

Các họ kiến trúc backbone (Backbone Architecture Families)

PHỤ THUỘC C/12-co-che-huan-luyen (toàn chương — không có nó thì mọi kiến trúc ở đây chỉ là sơ đồ khối); B/10-thiet-ke-kien-truc (trực đánh đổi giả định-dữ liệu); B/11-gioi-han-tinh-toan (FLOPs không phải độ trễ).

TÓM TẮT

Chương này đọc lịch sử kiến trúc như *một cuộc tranh luận kéo dài về việc nên cài bao nhiêu giả định vào mô hình*. Mỗi thể hệ trả lời khác nhau, vì lượng dữ liệu sẵn có ở thời điểm đó khác nhau. Đọc xong bạn chọn được **backbone** theo ràng buộc thật của mình — dữ liệu, độ trễ, độ phân giải — thay vì theo bảng xếp hạng, và đọc được cấu trúc một repo thị giác lạ trong mười phút. Nếu chỉ nhớ một điều: mọi kiến trúc trong chương là một điểm trên cùng một trục. Một đầu là “cài sẵn nhiều giả định, cần ít dữ liệu”, đầu kia là “cài ít giả định, cần nhiều dữ liệu”. Bài học đắt nhất: phần lớn chênh lệch từng được gán cho kiến trúc hoá ra thuộc về công thức huấn luyện.

KIẾN THỨC CẦN CÓ TRƯỚC

- **backbone** — phần mạng làm nhiệm vụ trích đặc trưng từ ảnh, dùng chung được cho nhiều bài toán; phần sinh đầu ra cụ thể được lắp thêm phía sau.
- **inductive bias** — tập giả định mà mô hình mang sẵn trước khi nhìn dữ liệu; nhiều giả định đúng thì học nhanh với ít mẫu, giả định sai thì chặn luôn nghiệm đúng.
- **convolution** — phép nhân một cửa sổ trọng số nhỏ trượt khắp ảnh; vì cùng bộ trọng số dùng cho mọi vị trí, nó cài sẵn hai giả định — quan hệ gần là quan trọng, và mẫu hình có ý nghĩa như nhau ở mọi vị trí.
- **trường tiếp nhận (receptive field)** — vùng pixel đầu vào ảnh hưởng tới một giá trị đầu ra; nó lớn dần theo chiều sâu trong CNN và bằng toàn ảnh ngay từ tầng đầu trong attention toàn cục.
- **attention** — cơ chế cho mỗi vị trí tự chọn lấy thông tin từ mọi vị trí khác bằng trọng số học được, thay vì chờ thông tin lan dần qua từng tầng.
- **FLOPs** — số phép nhân-cộng mà mô hình thực hiện; nó là chỉ số đại diện phổ biến cho chi phí, nhưng *không* tỉ lệ với độ trễ thật vì còn phụ thuộc băng thông bộ nhớ.
- **pretraining** — huấn luyện trước trên một tập rất lớn để học đặc trưng chung, rồi mới huấn luyện tiếp trên tập nhỏ của bài toán đích.

CÂU HỎI MỞ ĐẦU

- Q1. Nếu **attention** cho mọi vị trí nói chuyện trực tiếp với nhau ngay từ tầng đầu tiên, vì sao suốt mười năm người ta vẫn dùng **convolution** — thứ cổ tình chỉ cho phép nhìn một cửa sổ 3×3 ?
- Q2. Một mạng phẳng 56 tầng có sai số *huấn luyện* cao hơn mạng 20 tầng, dù nó thừa sức chứa để bắt chước mạng nông bằng cách để các tầng thừa làm ánh xạ đồng nhất. Chuyện gì đã xảy ra?
- Q3. MobileNet giảm số phép nhân khoảng 8,7 lần. Vì sao độ trễ đo được trên GPU lại không giảm 8,7 lần?
- Q4. Nếu ConvNeXt cho thấy CNN vẫn ngang ViT khi dùng cùng công thức huấn luyện, thì mười năm “cải tiến kiến trúc” đã thực sự cải tiến cái gì?

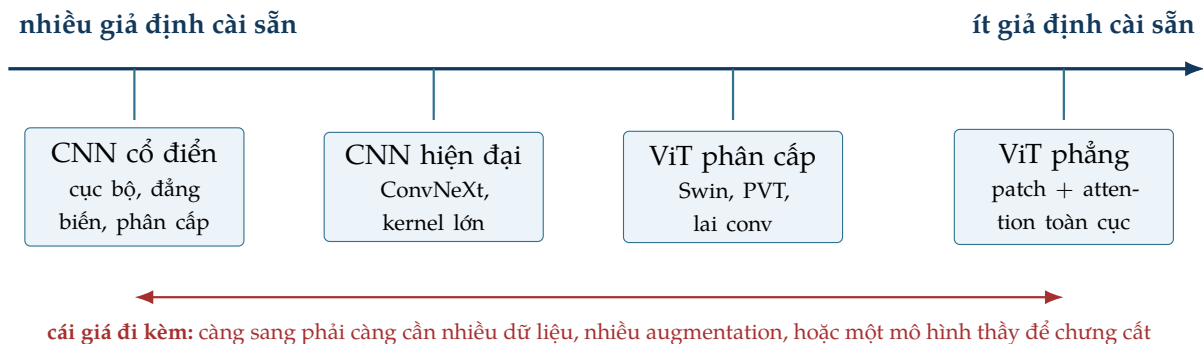
Q5. Vì sao cùng một kiến trúc lại thua CNN khi có một triệu ảnh và thắng khi có ba trăm triệu ảnh — điều gì trong mô hình thay đổi theo lượng dữ liệu?

1.1 Vì sao chương này chia làm ba mạch

Ba mục của chương không phải ba họ kiến trúc song song mà là ba lần trả lời cùng một câu hỏi, trên ba loại dữ liệu và ở ba thời điểm khác nhau. Đọc theo thứ tự này thì mục sau luôn có sẵn khái niệm mà mục trước dựng lên.

1. **Mạch 1 — dòng CNN.** Bắt đầu từ câu trả lời “cài nhiều giả định”, rồi theo dõi mười lăm năm gõ từng nút thắt. Mục này dựng lên các khái niệm nền của cả chương: trường tiếp nhận, chi phí tính toán, kết nối tắt, và bài học ConvNeXt về recipe.
2. **Mạch 2 — từ chuỗi tới attention.** Chuyển sang loại dữ liệu mà convolution không phục vụ được — dữ liệu có thứ tự và quan hệ xa — và theo dõi chuỗi RNN, LSTM, attention, transformer, rồi các cách chống chi phí bậc hai. Đây là mục cung cấp cơ chế cho mạch 3.
3. **Mạch 3 — ViT và inductive bias.** Mang attention vào ảnh, tức chọn câu trả lời “cài rất ít giả định”, rồi đo cái giá. Mục này chỉ đọc được sau hai mục trước, vì nó liên tục so sánh với cả hai.

Hình 1.1 đặt cả ba mạch lên một trục duy nhất. Điều đáng chú ý là trục này không có điểm “tốt nhất”: vị trí đúng phụ thuộc vào lượng dữ liệu bạn có, và đó chính là nội dung của cuộc tranh luận.



Hình 1.1. Bốn họ kiến trúc đặt trên trục “lượng giả định cài sẵn”. Trục này không có điểm tối ưu tuyệt đối: vị trí đúng phụ thuộc lượng dữ liệu có sẵn mà bạn nắm trong tay, nên cùng một bài toán có thể đổi câu trả lời khi tập dữ liệu lớn lên.

Bản đồ chương 13 — Các họ kiến trúc backbone

TÓM TẮT

Chương 13 gồm ba mục, và chúng là ba lần trả lời *cùng một câu hỏi*: nên cài sẵn bao nhiêu giả định vào mô hình. Ba lần đó diễn ra trên ba loại dữ liệu và ở ba thời điểm khác nhau. Thứ tự 1 → 2 → 3 là bắt buộc vì mục 3 liên tục so sánh với cả hai mục trước. Nếu chỉ nhớ một điều: mọi kiến trúc trong chương nằm trên một trục duy nhất giữa “nhiều giả định, ít dữ liệu” và “ít giả định, nhiều dữ liệu”, và không có điểm nào trên trục đó là tốt nhất một cách tuyệt đối.

KIẾN THỨC CẦN CÓ TRƯỚC

- **backbone** — phần mạng trích đặc trưng từ ảnh, dùng chung cho nhiều bài toán; phần sinh đầu ra cụ thể (nhân, hộp, mặt nạ) được lắp thêm phía sau.
- **inductive bias** — tập giả định mà mô hình mang sẵn trước khi nhìn dữ liệu; nó là đại lượng trung tâm mà cả chương xoay quanh.
- **convolution** — phép nhân một cửa sổ trọng số nhỏ trượt khắp ảnh, cài sẵn giả định “quan hệ gần là quan trọng” và “mẫu hình có ý nghĩa như nhau ở mọi vị trí”.
- **attention** — cơ chế cho mỗi vị trí lấy thông tin trực tiếp từ mọi vị trí khác bằng trọng số học được, không cần thông tin lan dần qua nhiều tầng.

Có gì trong chương này**Bảng 1.1.** Ba mục của chương 13: câu hỏi trung tâm, phụ thuộc và mức độ đầu tư.

Mục	Câu hỏi trung tâm	Phụ thuộc	Mức độ
1. Dòng CNN	Mỗi thể hệ CNN gỡ nút thắt nào, và đẩy độ khó sang trục nào?	C/12	Cốt lõi [T2]
2. Từ chuỗi tới attention	Xử lý quan hệ xa thế nào, và cái giá của việc rút ngắn đường đi thông tin là gì?	Mục 1, C/12/02	Cốt lõi [T2]
3. ViT và inductive bias	Vứt bỏ giả định thị giác thì mất gì, và trả lại bằng cách nào?	Mục 1 và 2	Cốt lõi [T2]

Vì sao lại chia như vậy

Cách chia phổ biến trong sách là chia theo *họ kiến trúc*: CNN, RNN, Transformer. Cách đó che mất điều quan trọng nhất. Ba họ này trả lời cùng một câu hỏi thiết kế, chứ không phục vụ ba thể giới tách biệt. Ở đây ta chia theo *loại dữ liệu và lượng giả định*: mục 1 là dữ liệu lưới với nhiều giả định; mục 2 là dữ liệu có thứ tự, nơi giả định lưới không dùng được; mục 3 là quay lại dữ liệu lưới nhưng với rất ít giả định.

Lợi ích của cách chia này lộ ra ở hai chỗ nổi. Thứ nhất, cổng của LSTM ở mục 2 và kết nối tắt của ResNet ở mục 1 hoá ra là cùng một mẹo — tạo một đường cộng cho gradient — và cách chia theo họ kiến trúc sẽ giấu mất điều đó. Thứ hai, mục 3 chỉ có nghĩa như một *thí nghiệm đối chứng* cho mục 1: nó đo giá trị của các giả định mà mục 1 mô tả, bằng đơn vị là lượng dữ liệu.

Đọc theo thứ tự nào

1. Đọc tuần tự 1 → 2 → 3. Mục 3 tham chiếu liên tục sang hai mục kia nên đọc trước sẽ hệt.
2. Nếu bạn chỉ cần chọn một backbone cho một dự án đang chạy: đọc mục 1 phần bảng dòng CNN, rồi nhảy tới mục 3 phần G (tách backbone, neck, head) và phần giới hạn. Quay lại mục 2 khi phải xử lý ảnh độ phân giải cao.
3. Nếu bạn đến từ phía SLAM hoặc hệ thống nhúng: đọc mục 1 phần C (ba phép tính tay) trước tiên, vì đó là chỗ FLOPs, cường độ số học và độ trễ thật gặp nhau.

Cốt lõi và tham khảo

Ba phần đáng quay lại nhiều lần: ba phép tính tay ở mục 1 (xếp chồng 3×3 , nút thắt 1×1 , depthwise separable); lập luận $\sqrt{d}$ và bảng ba hướng chống chi phí bậc hai ở mục 2; và trục giả định–dữ liệu cùng sơ đồ backbone–neck–head ở mục 3. Ba phần đọc lướt lần đầu rồi tra lại khi cần: danh sách biến thể convolution, chi tiết các họ mã hoá vị trí, và phần mô hình không gian trạng thái — phần cuối này thay đổi nhanh nhất nên cũng mất giá nhanh nhất.

Nối vào phần còn lại của cây

Chương này tiêu thụ toàn bộ C/12-co-che-huan-luyen và trực đánh đổi ở B/10-thiet-ke-kien-truc; nó cấp đầu vào cho C/14-bai-toan-cv-loi (head cụ thể lắp lên backbone), C/16-thi-giac-3d và C/18-pretraining-va-foundation. Hai nhánh ngang đáng nhớ: phần FLOPs và cường độ số học nổi sang B/11-gioi-han-tinh-toan cùng D/20-toi-uu-va-nen-mo-hinh; còn bài học ConvNeXt — rằng recipe giải thích phần lớn chênh lệch được gán cho kiến trúc — là tiền đề trực tiếp của E/24-thi-nghiem-va-ablation.

1.2 Dòng CNN: gỡ từng nút thắt (The CNN Lineage)

PHỤ THUỘC C/12-co-che-huan-luyen (đặc biệt mục 2 — kết nối tất là thành viên thứ tư của họ “giữ tích Jacobian gần 1”); B/11-gioi-han-tinh-toan (FLOPs, cường độ số học, chế độ nghiêng bộ nhớ).

TÓM TẮT

Mục này đọc mười lăm năm lịch sử *CNN* như một chuỗi gỡ nút thắt: mỗi kiến trúc ra đời để phá đúng một rào cản của kiến trúc trước, và tạo ra rào cản mới cho kiến trúc sau. Đọc xong bạn nhìn một *backbone* lạ và gọi tên được nó đang tối ưu cho ràng buộc nào: tham số, độ sâu, FLOPs, hay độ trễ thật. Nhờ đó bạn chọn backbone theo ràng buộc của mình, thay vì theo bảng xếp hạng. Nếu chỉ nhớ một điều: bài học lớn nhất của cả dòng nằm ở mắt xích cuối, ConvNeXt. Rất nhiều “cải tiến kiến trúc” trong lịch sử hoá ra là cải tiến *công thức huấn luyện*. Nên mọi so sánh kiến trúc không kiểm soát recipe đều đáng ngờ.

KIẾN THỨC CẦN CÓ TRƯỚC

- **convolution** — phép nhân một cửa sổ trọng số nhỏ, gọi là kernel hay filter, trượt khắp ảnh; vì cùng bộ trọng số dùng cho mọi vị trí nên số tham số không phụ thuộc kích thước ảnh.
- **kênh (channel)** — chiều thứ ba của một bản đồ đặc trưng — ảnh RGB có 3 kênh đầu vào, còn một tầng giữa mạng có thể có 256 kênh, mỗi kênh là một loại đặc trưng mà mạng học được.
- **stride** — bước nhảy của cửa sổ khi trượt; stride 2 làm bản đồ đặc trưng nhỏ đi một nửa mỗi chiều, nên nó là cách chính để giảm độ phân giải và tăng tầm nhìn.
- **trường tiếp nhận (receptive field)** — vùng pixel đầu vào ảnh hưởng tới một giá trị đầu ra; nó lớn dần theo chiều sâu, và cả mục này xoay quanh việc mở rộng nó cho rẻ.
- **pooling** — phép gộp một vùng nhỏ thành một giá trị (lấy lớn nhất hoặc lấy trung bình), dùng để giảm độ phân giải mà không tốn tham số.
- **FLOPs** — số phép nhân-cộng mà mô hình thực hiện; đây là chỉ số đại diện phổ biến cho chi phí, nhưng *không* tỉ lệ với độ trễ thật.
- **cường độ số học (arithmetic intensity)** — số phép tính thực hiện trên mỗi byte dữ liệu đọc từ bộ nhớ; thấp thì kernel bị nghẽn ở bảng thông chữ không ở khả năng tính toán, và giảm FLOPs lúc đó gần như vô ích.
- **recipe (công thức huấn luyện)** — toàn bộ các lựa chọn không thuộc kiến trúc — optimizer, lịch học, số epoch, augmentation — và bài học cuối mục là chúng giải thích phần lớn chênh lệch giữa các kiến trúc.

A. Đặt vấn đề

Toàn bộ chương 13 là một cuộc tranh luận kéo dài về một câu hỏi: *nên cài sẵn bao nhiêu giả định vào mô hình?* **Convolution** là một câu trả lời rất mạnh cho câu hỏi đó, và nó cài sẵn đúng ba giả định. Thứ nhất là **tính cục bộ (locality)** — một pixel chỉ liên quan trực tiếp tới các pixel lân cận, nên trọng số bằng 0 ở ngoài cửa sổ $k \times k$. Thứ hai là **chia sẻ trọng số (weight sharing)**, tương đương

với **đẳng biến tịnh tiến** (*translation equivariance*) — một mẫu hình có ý nghĩa như nhau dù nó nằm ở đâu trong ảnh. Thứ ba là **phân cấp** (*hierarchy*) — đặc trưng phức tạp được ghép từ đặc trưng đơn giản, nên tăng dần độ trừu tượng và giảm dần độ phân giải.

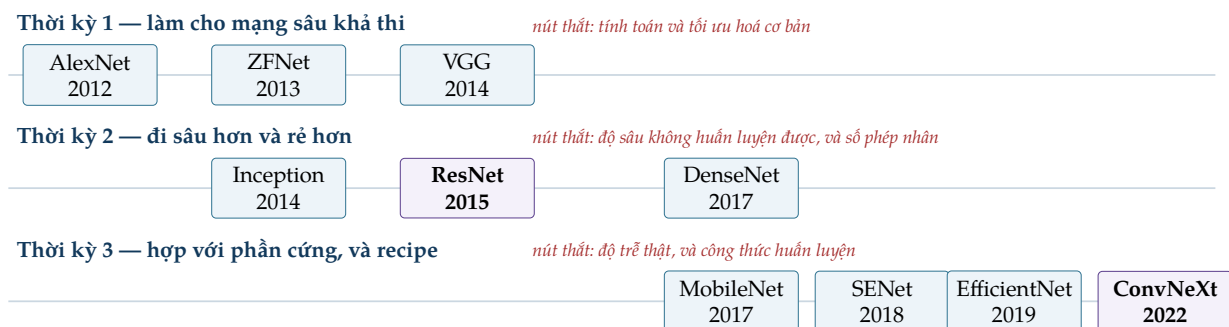
Ba giả định này đến từ vật lý của việc tạo ảnh chứ không từ toán học, và chúng đúng đủ tốt để một mạng conv cần ít dữ liệu hơn hẳn một mạng dày đặc cùng sức chứa. Nhưng chúng cũng khoá lại một không gian thiết kế, và lịch sử CNN chính là lịch sử của việc lần lượt phát hiện ra rào cản nào trong không gian đó đang chặn, rồi gỡ nó. Giá trị cụ thể của mục này: cầm một backbone lạ lên, bạn trả lời được ba câu mà không cần chạy thử. Nó đang tiết kiệm cái gì? Nó trả giá ở đâu? Và ràng buộc nào khiến người ta chọn nó?

TRỰC GIÁC

Đừng đọc dòng CNN như một cuộc thi xem ai chính xác hơn. Hãy đọc nó như lịch sử tối ưu một đường ống sản xuất, nơi mỗi thế hệ tìm ra *nút cổ chai hiện thời* rồi mở rộng đúng chỗ đó. Mở đúng nút cổ chai thì thông lượng tăng vọt, và nút cổ chai lập tức chuyển sang chỗ khác. Đó là lý do mỗi bước tiến trong bảng dưới đây lại sinh ra một vấn đề mới ở trực khác. Người đọc lịch sử theo cách này đoán được bước tiếp theo; người đọc theo cách “ai chính xác hơn” chỉ nhớ được tên.

B. Dòng thời gian: ba thời kỳ, ba nút thắt khác nhau

Nhìn từ xa, mười lăm năm này chia làm ba thời kỳ rõ rệt, và *nút thắt của mỗi thời kỳ nằm ở một trực khác nhau*. Thời kỳ đầu vướng ở tính khả thi. Thời kỳ giữa vướng ở khả năng tối ưu hoá. Thời kỳ sau vướng ở phần cứng và ở công thức huấn luyện.



Hình 1.2. Dòng CNN đặt trên trục thời gian và tách theo nút thắt được gỡ. Hai mắt xích được tô đậm vì chúng thay đổi cách ngành suy nghĩ chứ không chỉ thay đổi con số: ResNet cho thấy rào cản của mạng sâu là *tối ưu hoá* chứ không phải sức chứa, còn ConvNeXt cho thấy phần lớn khoảng cách giữa các kiến trúc là khoảng cách công thức huấn luyện.

Bảng 1.2 đọc cùng Hình 1.2 theo chiều dọc; cột cuối là cột đáng nhớ, vì nó cho thấy không bước nào miễn phí.

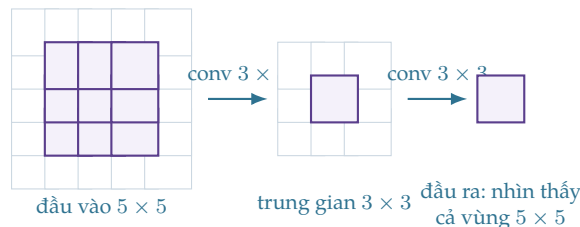
Bảng 1.2. Dòng CNN dọc theo mạch gỡ nút thắt. Cột cuối cho thấy mỗi bước tiến đẩy độ khó sang một trục khác chứ không xoá nó.

Bước	Gỡ nút thắt gì	Cái giá / vấn đề mới
LeNet	Chứng minh conv + pooling + huấn luyện gradient học được đặc trưng thị giác [1]	Chỉ chạy ở quy mô chữ số; chưa có dữ liệu và phần cứng để mở rộng
AlexNet	Chứng minh mạng sâu + GPU + ReLU + dropout là khả thi ở quy mô ImageNet [2]	Kiến trúc làm bằng tay, filter lớn 11×11 , rất nhiều tham số ở tầng FC
ZFNet	Trực quan hoá để thấy mạng học gì, từ đó chỉnh filter và stride	Đóng góp là phương pháp quan sát, không phải kiến trúc mới
VGG	Xếp chồng 3×3 : cùng trường tiếp nhận, ít tham số hơn, nhiều phi tuyến hơn [3]	Tầng FC chiếm phần lớn tham số; tổng FLOPs rất cao
Inception	Nhiều tỉ lệ song song; 1×1 làm nút thắt giảm chiều trước phép tính nặng [4], [5]	Kiến trúc phức tạp, nhiều siêu tham số nhánh, khó chỉnh và khó mở rộng
ResNet	Vấn đề <i>tối ưu hoá</i> của mạng sâu, bằng đường cộng đồng nhất [6]	Vẫn nhiều FLOPs; nhánh tắt buộc giữ thêm kích hoạt nên tốn bộ nhớ hơn
DenseNet	Tái dùng đặc trưng tối đa, ít tham số hơn ở cùng độ chính xác [7]	Phép nối làm bộ nhớ và băng thông phình ra nếu cài đặt ngây thơ
MobileNet	Chi phí tính toán trên thiết bị yếu, bằng depthwise separable [8]	Giảm FLOPs nhiều hơn giảm độ trễ thật, vì cường độ số học tụt
SENet	Mạng không tự điều chỉnh được tầm quan trọng giữa các kênh [9]	Thêm một điểm đồng bộ toàn cục; lợi ích nhỏ dần khi backbone đã mạnh
EfficientNet	Việc mở rộng mô hình bị làm tùy tiện theo một chiều [10]	Tối ưu cho FLOPs, không tối ưu cho một phần cứng cụ thể
ConvNeXt	Nghi vấn “ViT thắng nhờ kiến trúc”, bằng cách hiện đại hoá recipe của CNN [11]	Cho thấy phần lớn chênh lệch trước đó là do công thức huấn luyện, làm mọi so sánh kiến trúc cũ trở nên đáng ngờ

C. Ba phép tính tay giải thích ba bước quan trọng nhất

Ba bước có ảnh hưởng lớn nhất trong bảng đều rút gọn được thành một phép tính ngắn. Tự làm ba phép tính này một lần thì bạn nhớ chúng mãi.

VGG — vì sao xếp chồng 3×3 . Hai tầng 3×3 nối tiếp có cùng trường tiếp nhận với một tầng 5×5 ; ba tầng thì bằng 7×7 . Nhưng chi phí tham số khác hẳn. Với C kênh vào và ra, một tầng 5×5 tốn $25C^2$ tham số, còn hai tầng 3×3 chỉ tốn $2 \cdot 9C^2 = 18C^2$ — ít hơn 28 phần trăm. So 7×7 với ba tầng 3×3 thì là $49C^2$ đối với $27C^2$, ít hơn 45 phần trăm. Và ta còn được thêm hai lần phi tuyến. Nói gọn: cùng tầm nhìn, ít tham số hơn, nhiều khả năng biểu diễn hơn — một trong những phép đổi hiểm hơi tốt ở cả ba mặt.

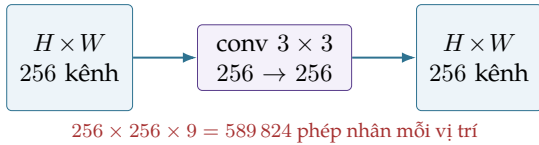


Hình 1.3. Hai tầng 3×3 nối tiếp cho một ô đầu ra nhìn thấy đúng vùng 5×5 của đầu vào. Cùng trường tiếp nhận với một tầng 5×5 , nhưng chỉ tốn $18C^2$ tham số thay vì $25C^2$. Và có thêm một lần phi tuyến ở giữa.

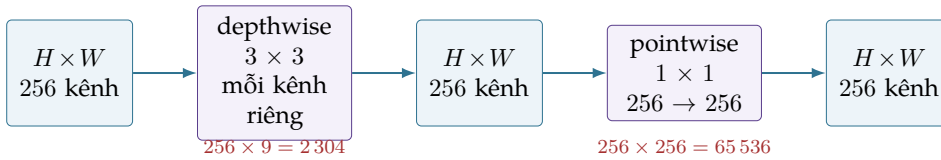
Hình 1.3 là toàn bộ lập luận của VGG dưới dạng hình. Điều nó *không* nói ra là nút thắt kế tiếp: tầng kết nối đầy đủ đầu tiên của VGG-16 có $7 \cdot 7 \cdot 512 \cdot 4096 \approx 103$ triệu tham số, tức khoảng ba phần tư toàn bộ mô hình nằm trong một tầng duy nhất không phải conv. Đó là lý do các thế hệ sau thay tầng FC bằng *global average pooling*.

Inception — vì sao 1×1 là nút thắt hiệu quả. Giả sử cần một conv 3×3 từ 256 kênh sang 256 kênh. Chi phí trực tiếp là $256 \cdot 256 \cdot 9 = 589\,824$ phép nhân-cộng cho mỗi vị trí không gian. Bây giờ chèn hai tầng 1×1 : giảm $256 \rightarrow 64$, làm 3×3 trong không gian 64 kênh, rồi mở lại $64 \rightarrow 256$. Tổng chi phí là $256 \cdot 64 + 64 \cdot 64 \cdot 9 + 64 \cdot 256 = 16\,384 + 36\,864 + 16\,384 = 69\,632$, tức rẻ hơn 8,5 lần. Ý nghĩa của phép tính: phần đắt nhất của conv là tương tác giữa *mọi* kênh vào với *mọi* kênh ra, nhân với k^2 ; nếu ta làm phép tính không gian ở một không gian kênh hẹp thì cái k^2 chỉ nhân với một số nhỏ. Đây cũng chính là khối *bottleneck* của ResNet, và là ý tưởng nền của phần lớn kiến trúc hiệu quả về sau.

conv 3×3 thường: mọi kênh vào nói chuyện với mọi kênh ra, ở mọi vị trí



depthwise separable: tách phần không gian khỏi phần kênh



Tổng = 67 840, tức $\frac{1}{C_{\text{out}}} + \frac{1}{k^2} = \frac{1}{256} + \frac{1}{9} \approx 0,115$ lần chi phí gốc — rẻ hơn khoảng 8,7 lần.

Nhưng bước depthwise đọc mỗi phần tử để làm vòn vẹn 9 phép nhân, nên cường độ số học của nó tụt xuống mức nghen bằng thông.

Hình 1.4. Vì sao depthwise separable rẻ, và vì sao “rẻ” theo FLOPs không có nghĩa là “nhanh”. Conv thường trả tiền cho tích của ba thứ cùng lúc: kênh vào, kênh ra, và diện tích của số. Phép tách chia nó thành hai bước, mỗi bước chỉ trả cho *hai* trong ba thứ đó — đây là toàn bộ nguồn gốc của tỉ số $1/C_{\text{out}} + 1/k^2$. Cái giá nằm ở dòng cuối: bước depthwise có rất ít phép tính trên mỗi byte đọc vào, nên nó rơi vào phía nghen bộ nhớ và thời gian thực tế không giảm theo tỉ lệ FLOPs.

Hình 1.4 bày ra cả hai vế của phép đổi trên cùng một sơ đồ, và đó là lý do nó đáng vẽ: con số 8,7 và lời cảnh báo về độ trễ luôn phải đọc cùng nhau.

MobileNet — và vì sao FLOPs không phải độ trễ. Depthwise separable tách một conv thành hai: một conv $k \times k$ chạy độc lập trên từng kênh, rồi một conv 1×1 trộn kênh. Tỉ số chi phí so với conv thường là $\frac{1}{C_{\text{out}}} + \frac{1}{k^2}$; với $k = 3$ và $C_{\text{out}} = 256$ thì bằng $0,0039 + 0,111 \approx 0,115$, tức rẻ hơn khoảng 8,7 lần. Nhưng độ trễ thực đo được hầu như không bao giờ giảm 8,7 lần, và lý do là thứ bạn đã gặp khi tối ưu CUDA: phần depthwise đọc mỗi phần tử đầu vào để làm vòn vẹn $k^2 = 9$ phép nhân, nên **cường độ số học** (*arithmetic intensity*) của nó cực thấp và nó rơi vào chế độ nghen bằng thông bộ nhớ. Giảm FLOPs của một kernel vốn đã nghen ở bộ nhớ thì gần như không giảm thời gian. Đây là ví dụ rõ nhất trong cả cây về việc một chỉ số đại diện — FLOPs — đo sai thứ mà ta quan tâm.

↪ **ràng buộc bởi B/11** — mô hình roofline nói chính xác khi nào giảm FLOPs không giảm thời gian: khi kernel đã nằm ở phía nghen bằng thông

D. ResNet: nút thắt là tối ưu hoá, không phải sức chứa

Đóng góp của ResNet hay bị kể sai thành “mạng sâu hơn thì mạnh hơn”. Bằng chứng gốc nói điều khác và mạnh hơn nhiều: một mạng phẳng 56 tầng có *sai số huấn luyện* cao hơn một mạng phẳng 20 tầng [6]. Sai số huấn luyện, không phải sai số kiểm thử. Nên đây không phải overfitting, và cũng không phải thiếu sức chứa. Mạng sâu hơn về nguyên tắc biểu diễn được mọi thứ mạng

nông biểu diễn được: chỉ cần các tầng thừa làm ánh xạ đồng nhất. Vấn đề là gradient descent không tìm ra nghiệm đó.

Kết nối tắt $y = x + F(x)$ làm hai việc cùng lúc, và cả hai đều đã xuất hiện ở chương 12. Về mặt biểu diễn, ánh xạ đồng nhất trở thành phương án *mặc định* — chỉ cần $F \approx 0$, điều mà khởi tạo nhỏ cho sẵn. Về mặt gradient, Jacobian của khối là $I + \partial F / \partial x$, nên tích các Jacobian qua nhiều khối luôn còn một thành phần bằng 1 không thể triệt tiêu; đây chính xác là thành viên thứ tư của họ kỹ thuật “giữ ρ gần 1” mà mục C/12/02 đã mở ngoặc để dành.

Khối phẳng



$\frac{\partial y}{\partial x} = J_2 J_1$ — mọi thừa số đều có thể nhỏ hơn 1
đường tắt: x đi thẳng, không qua phép toán nào

Khối residual



Hệ quả kép: ánh xạ đồng nhất trở thành phương án *mặc định* (chỉ cần $F \approx 0$), và gradient luôn có một đường về không bị nhân dồn.

$\frac{\partial y}{\partial x} = I + \frac{\partial F}{\partial x}$ — số hạng I không bao giờ triệt tiêu

Hình 1.5. Kết nối tắt sửa vấn đề gì, nhìn từ phía lượt ngược. Ở khối phẳng, đạo hàm của đầu ra theo đầu vào là một tích thuần tuý, nên nếu mỗi thừa số hơi nhỏ hơn 1 thì sau vài chục khối nó tắt — đúng cơ chế ở C/12/02. Ở khối residual, đường cộng thêm một số hạng I vào Jacobian, và số hạng đó không phụ thuộc trọng số nên không thể bị nhân cho tiêu biến. Đây là lý do bằng chứng gốc của ResNet là sai số *huấn luyện* chứ không phải sai số kiểm thử: thứ được sửa là khả năng tối ưu hoá, không phải sức chứa.

Hình 1.5 cũng cho thấy cái giá đã nhắc ở bảng: mũi tên đường tắt buộc tensor x phải sống tới tận điểm cộng, nên bộ nhớ kích hoạt của một mạng residual cao hơn một mạng phẳng cùng độ sâu.

Cái giá thì cụ thể: nhánh tắt buộc giữ tensor x sống tới tận chỗ cộng, nên bộ nhớ kích hoạt tăng so với một mạng phẳng cùng độ sâu.

↔ cùng nguyên lý với C/12/02 — kết nối tắt, hàm kích hoạt, khởi tạo và chuẩn hoá đều nhắm vào một đại lượng: hệ số nhân của tích các Jacobian

E. Các biến thể convolution và mục đích riêng

Đây là một bộ sưu tập chứ không phải một chuỗi, nên đọc theo kiểu tra cứu. Điểm chung: mỗi biến thể *nói lỏng hoặc siết chặt* đúng một trong ba giả định gốc ở mục A.

- **Depthwise separable** — tách tương tác không gian khỏi tương tác kênh.
 - **Được:** chi phí giảm khoảng k^2 lần, như phép tính ở mục C.
 - **Mất:** cường độ số học tụt nên độ trễ thật giảm ít hơn nhiều; và giả định “không gian và kênh tách rời được” không phải lúc nào cũng đúng.
- **Group convolution** — chia kênh thành g nhóm, mỗi nhóm chỉ nói chuyện trong nội bộ. Ra đời từ ràng buộc bộ nhớ của AlexNet trên hai GPU, sau đó thành công cụ giảm tham số. **Mất:** thông tin không đi qua ranh giới nhóm, nên cần một phép trộn kênh (ví dụ shuffle hoặc 1×1) để bù.

- **Dilated convolution** [12] — giãn khoảng cách giữa các điểm lấy mẫu trong cửa sổ, nên trường tiếp nhận tăng theo cấp số nhân mà không cần giảm độ phân giải. Sinh ra cho segmentation, nơi giảm độ phân giải là mất mát trực tiếp. *Mất*: hiện tượng lưới — các ô lấy mẫu không phủ kín nên xuất hiện hoa văn răng lược nếu xếp chồng nhiều tầng cùng hệ số giãn.
- **Deformable convolution** [13] — học luôn cả *vị trí* lấy mẫu thay vì cố định trên lưới, tức nói lỏng giả định cục bộ theo một cách có điều khiển. *Mất*: truy cập bộ nhớ không đều nên rất khó tối ưu trên GPU, và là một trong những lý do phổ biến khiến một mô hình không xuất được sang runtime suy luận.
- **Transposed convolution** — tăng độ phân giải ở nhánh giải mã. *Mất*: hiện tượng bàn cờ khi kích thước kernel không chia hết cho stride; nhiều cài đặt hiện đại thay nó bằng *upsample rồi conv thường* để tránh hẳn.

F. ConvNeXt và bài học đắt nhất của cả mục

Khi ViT vượt CNN trên ImageNet, cách đọc tự nhiên là “*attention* tốt hơn convolution”. ConvNeXt kiểm chứng cách đọc đó bằng một thí nghiệm có kỷ luật. Nhóm tác giả giữ nguyên convolution, rồi lần lượt áp từng thành phần của recipe hiện đại và đo sau mỗi bước [11]. Danh sách gồm lịch học dài hơn, AdamW, augmentation mạnh, stochastic depth, LayerNorm thay Batch-Norm, kernel lớn hơn, và tỉ lệ tăng theo kiểu transformer. Kết quả là một CNN thuần bắt kịp ViT cùng hạng.

Kết luận đúng không phải “CNN thắng” mà là: *phần lớn khoảng cách được gán cho kiến trúc thực ra thuộc về công thức huấn luyện*. Điều này thay đổi cách bạn đọc mọi paper kiến trúc về sau. Câu hỏi đầu tiên khi thấy một bảng so sánh không còn là “kiến trúc nào cao hơn”. Nó thành “hai dòng này có cùng recipe không, và nếu không thì ai được tune kỹ hơn”. Đây cũng là lý do chương E/24-thi-nghiem-va-ablation tồn tại.

↪ **hệ quả của E/24** — một bảng so sánh chỉ có nghĩa khi hai dòng được huấn luyện bằng cùng công thức và được tune ngang nhau

G. Giới hạn và trường hợp hồng

Ba giới hạn đáng mang theo. Thứ nhất, *FLOPs không phải độ trễ, và số tham số cũng không*. Ba đại lượng này tương quan yếu với nhau và với thời gian chạy thật; MobileNet và DenseNet là hai ví dụ ngược chiều nhau về đúng điểm này. Bất kỳ lựa chọn backbone nào cũng phải đo trên đúng phần cứng và đúng chế độ batch mà bạn sẽ triển khai.

Thứ hai, *độ chính xác ImageNet là một chỉ số đại diện, không phải mục tiêu*. Tương quan giữa nó và chất lượng ở tác vụ khác là có thật, nhưng không chặt. Tương quan đó yếu đi rõ rệt với các tác vụ dự đoán dày như segmentation. Ở đó, độ phân giải và đặc trưng đa tỉ lệ quan trọng hơn khả năng phân loại toàn ảnh.

Thứ ba, các con số trong mục này thuộc hai loại khác nhau. Các phép tính tham số và FLOPs ở mục C là số học thuần túy nên chính xác. Ngược lại, các phát biểu kiểu “rẻ hơn 8,7 lần nhưng độ trễ giảm ít hơn nhiều” là mô tả định tính của một hiện tượng đã được quan sát rộng rãi; mức giảm cụ thể phụ thuộc GPU, thư viện và kích thước tensor tới mức tôi không nêu một con số nào ở đây.

CẠM BẤY

Cạm bẫy phổ biến nhất khi chọn backbone là so sánh hai mô hình được huấn luyện bằng hai recipe khác nhau rồi kết luận về kiến trúc. Một ResNet-50 theo recipe 2015 và một ResNet-50 theo recipe 2022 chênh nhau vài điểm phần trăm trên ImageNet. Khoảng cách

đó lớn hơn khoảng cách giữa nhiều cặp kiến trúc mà người ta viết hẳn một paper để so. Trước khi tin bất kỳ bảng nào, hãy kiểm tra ba thứ: số epoch, bộ augmentation, và optimizer cùng lịch học.

CẦN NHỚ

Mỗi bước trong dòng CNN gỡ đúng một nút thắt và đẩy độ khó sang một trục khác: tham số $\rightarrow$ khả năng tối ưu hoá $\rightarrow$ chi phí tính toán $\rightarrow$ độ trễ trên phần cứng thật $\rightarrow$ công thức huấn luyện. Khi cầm một backbone mới, hỏi nó đang đứng ở nút nào trong chuỗi đó — câu trả lời cho bạn biết nó sẽ hỏng ở đâu. [T2]

H. Kết nối

Mục này giả định toàn bộ C/12-co-che-huan-luyen, và trả lại cho chương đó thành viên thứ tư của họ “giữ tích Jacobian gần 1”. Nó đi thẳng vào C/13/02-tu-chuoi-toi-attention, nơi câu hỏi “bao nhiêu giả định là đủ” được đặt cho dữ liệu có thứ tự. Rồi vào C/13/03-vit-va-inductive-bias, nơi ba giả định ở mục A bị gỡ bỏ có chủ đích để xem chuyện gì xảy ra. Phần FLOPs và cường độ số học nối trực tiếp sang B/11-gioi-han-tinh-toan và D/20-toi-uu-va-nen-mo-hinh; còn bài học ConvNeXt là tiền đề của E/24-thi-nghiem-va-ablation. Nếu bạn quan tâm việc lắp backbone vào một hệ thống thật, C/14-bai-toan-cv-loi là chương nói về phần neck và head gắn lên nó.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Ba giả định mà convolution cài sẵn là gì, và mỗi biến thể ở mục E nói lỏng giả định nào?
2. Vì sao xếp chồng hai tầng 3×3 tốt hơn một tầng 5×5 ở cả ba mặt, và trong tình huống nào nó không tốt hơn?
3. Bằng chứng gốc của ResNet là sai số *huấn luyện* chứ không phải kiểm thử. Vì sao chi tiết đó thay đổi hoàn toàn kết luận rút ra?
4. MobileNet giảm FLOPs khoảng 8,7 lần nhưng độ trễ giảm ít hơn nhiều — giải thích bằng cường độ số học thì thế nào, và phép đo nào xác nhận chẩn đoán đó?
5. Nếu ConvNeXt đúng khi nói recipe quan trọng hơn kiến trúc, điều đó thay đổi thứ tự bạn đọc một paper kiến trúc thế nào?
6. Deformable convolution mạnh hơn conv thường ở đâu, và vì sao nó thường bị loại khi triển khai lên thiết bị nhúng?

Nguồn

- [1] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [3] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [4] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [5] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [6] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [7] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [8] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [9] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [10] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [11] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [12] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [13] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [14] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [15] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [16] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [17] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [18] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [19] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [20] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [21] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [22] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.

- [23] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.

1.3 Từ chuỗi tới attention (From Sequences to Attention)

PHỤ THUỘC C/12/02-lam-cho-training-chay-duoc (tích các Jacobian — vanishing gradient ở đây là đúng hiện tượng đó, chỉ trải theo thời gian); C/13/01-dong-cnn (khái niệm trường tiếp nhận và inductive bias).

TÓM TẮT

Mục này đi từ mạng hồi quy tới **attention** và tới các mô hình không gian trạng thái, đọc như một chuỗi gỡ nút thắt chứ không như một danh sách kiến trúc. Đọc xong bạn giải thích được vì sao **transformer** bỏ được tính tuần tự, và vì sao nó phải trả bằng chi phí bậc hai. Bạn cũng nói được ba hướng tấn công chi phí đó khác nhau ở tầng nào: toán, kiến trúc, hay hệ thống. Nếu chỉ nhớ một điều: LSTM và ResNet giải cùng một bài toán bằng cùng một mẹo. Cả hai tạo một **đường cộng** để gradient đi qua mà không bị nhân dồn. Nhận ra điều đó thì hai kiến trúc trông rất khác nhau trở thành một ý tưởng duy nhất.

KIẾN THỨC CẦN CÓ TRƯỚC

- **chuỗi (sequence)** — dữ liệu có thứ tự, ví dụ một câu gồm nhiều từ hoặc một video gồm nhiều khung hình; “thứ tự là thông tin” nghĩa là hoán vị các phần tử sẽ đổi ý nghĩa.
- **token** — một phần tử của chuỗi sau khi đã được cắt và mã hoá thành vectơ số — một từ, một mảnh từ, hoặc với ảnh là một ô vuông 16×16 pixel.
- **embedding** — vectơ số biểu diễn một token; các token có nghĩa gần nhau được đặt gần nhau trong không gian này, và toàn bộ mô hình chỉ làm việc với vectơ chứ không với ký tự.
- **trạng thái ẩn (hidden state)** — vectơ mà mạng hồi quy mang theo từ bước thời gian này sang bước sau; nó là toàn bộ “trí nhớ” của mạng, nên mọi thứ không nhét được vào nó là mất.
- **BPTT (backpropagation through time)** — cách áp lan truyền ngược cho mạng hồi quy: trải mạng ra thành T bản sao theo thời gian rồi lan truyền ngược qua cả T bước, nên một chuỗi dài 100 hành xử như một mạng sâu 100 tầng.
- **attention** — cơ chế cho mỗi vị trí tự chọn lấy thông tin từ mọi vị trí khác bằng một trọng số học được, thay vì phải chờ thông tin truyền dần qua từng bước.
- **độ phức tạp bậc hai** — chi phí tỉ lệ với T^2 khi độ dài chuỗi là T ; gấp đôi độ dài thì chi phí gấp bốn, nên đây là rào cản chính khi mang attention sang ảnh độ phân giải cao.
- **HBM** — bộ nhớ băng thông cao gắn ngoài chip GPU — rất lớn nhưng chậm hơn nhiều so với bộ nhớ trên chip; số lần đọc ghi HBM thường quyết định tốc độ hơn cả số phép nhân.

A. Đặt vấn đề

Convolution giả định dữ liệu nằm trên một lưới đều và quan hệ quan trọng là quan hệ gần. Nhưng nhiều dữ liệu có *thứ tự* chứ không có lưới, và quan hệ quan trọng có thể ở rất xa: từ đầu câu quyết định nghĩa từ cuối câu, khung hình đầu video quyết định cách hiểu khung hình cuối. Câu hỏi của mục này là: cấu trúc nào xử lý được loại quan hệ đó, và mỗi câu trả lời phải trả giá bằng gì?

Giá trị cụ thể của mục: nhìn một kiến trúc xử lý chuỗi, bạn trả lời được ba câu. Thông tin đi từ vị trí i tới vị trí j qua bao nhiêu bước? Chi phí tăng theo độ dài như thế nào? Và cái gì trong kiến trúc đang giữ vai trò “biết thứ tự”? Ba câu đó đủ để phân loại mọi biến thể mà bạn sẽ gặp.

TRỰC GIÁC

Hãy đo mọi kiến trúc trong mục này bằng một đại lượng duy nhất: *độ dài đường đi của thông tin* giữa hai vị trí xa nhau. Trong mạng hồi quy, thông tin phải đi qua T trạm nối tiếp, và mỗi trạm nhân nó với một ma trận. Giống như truyền một tin nhắn qua dây chuyền thì thầm dài 100 người. Trong attention, mọi vị trí nói chuyện trực tiếp với mọi vị trí, đường đi bằng 1 — giống như thay dây chuyền thì thầm bằng một phòng họp mà ai cũng nghe thấy ai. Toàn bộ ưu điểm lẫn toàn bộ chi phí của transformer nằm gọn trong phép đổi đó: phòng họp không tam sao thất bản, nhưng số cặp người phải nghe nhau tăng theo bình phương số.

B. Mạch từ RNN tới state space model

Sáu bước dưới đây là một chuỗi liên tục, và mỗi bước chỉ hiểu được khi đã thấy vấn đề mà bước trước để lại.

I Mạch 2 — từ chuỗi tới attention

1 Vấn đề dữ liệu có thứ tự, nhưng mạng dày đặc và convolution đều không có khái niệm “bước trước”.

Giải pháp **mạng hồi quy** (*recurrent neural network, RNN*): giữ một trạng thái ẩn $h_t = f(Wh_{t-1} + Ux_t)$ và cập nhật nó theo từng bước, nên cùng một bộ trọng số dùng lại cho mọi độ dài chuỗi.

Vấn đề mới huấn luyện bằng BPTT nghĩa là lan truyền ngược qua T tầng dùng chung trọng số. Gradient vì thế tỉ lệ với lũy thừa T của cùng một ma trận. Đó đúng là hiện tượng vanishing và exploding của mục C/12/02, nhưng tệ hơn: thừa số lặp lại y hệt nhau nên không có triết tiêu ngẫu nhiên nào cứu.

2 Vấn đề trí nhớ dài không tồn tại: sau vài chục bước tín hiệu đã tắt.

Giải pháp **LSTM** [14] và GRU dùng **cổng** (*gate*) để tạo một trạng thái ô $c_t = f_t \odot c_{t-1} + i_t \odot g_t$.

Hệ quả đạo hàm $\partial c_t / \partial c_{t-1} = f_t$, nên khi cổng quên mở gần 1 thì gradient chảy qua mà không bị nhân với ma trận trọng số. Đây chính xác là mẹo của kết nối tắt trong ResNet, chỉ khác là hệ số của đường cộng ở đây học được thay vì cố định bằng 1.

Vấn đề mới tính tuần tự vẫn còn nguyên: h_t cần h_{t-1} , nên không song song hoá được theo thời gian, và độ dài dùng được vẫn giới hạn.

3 Vấn đề tuần tự nghĩa là không tận dụng được GPU, và thông tin xa vẫn phải đi qua mọi trạm trung gian.

Giải pháp **attention**. Mỗi vị trí sinh một truy vấn q ; mọi vị trí sinh khoá k và giá trị v . Đầu ra là tổ hợp $\text{softmax}(qK^\top / \sqrt{d})V$, tức truy cập trực tiếp mọi vị trí trong một bước. **Transformer** [15] bỏ hẳn phần hồi quy và chỉ giữ attention cộng mạng truyền thẳng theo vị trí, nên toàn bộ chuỗi được xử lý song song.

Giá phải trả chi phí và bộ nhớ bậc hai theo độ dài, và mất hoàn toàn khái niệm thứ tự vì phép tính bất biến với hoán vị.

4 Vấn đề mô hình không còn biết token nào đứng trước token nào.

Giải pháp **mã hoá vị trí** (*positional encoding*): cộng hoặc trộn thêm thông tin vị trí vào embedding.

Giả định vị trí có thể mã hoá thành một vectơ mà mô hình học cách dùng.

Hệ quả bốn họ với đánh đối khác nhau — tuyệt đối dạng sin, tuyệt đối học được, tương đối, và RoPE quay embedding theo góc tỉ lệ vị trí [16]. Tuyệt đối đơn giản nhưng ngoại suy kém ra ngoài độ dài đã thấy khi huấn luyện; tương đối và RoPE ngoại suy tốt hơn vì chúng mã hoá *khoảng cách* chứ không mã hoá chỉ số. Với ảnh còn cần biến thể hai chiều, vì vị trí là một cặp (x, y) chứ không phải một số.

5 Vấn đề chi phí bậc hai chặn đường tới ảnh độ phân giải cao.

Giải pháp ba hướng tấn công ở ba tầng khác nhau — xấp xỉ toán học, giới hạn phạm vi kiến trúc, và tối ưu truy cập bộ nhớ — xem Bảng 1.3.

Hệ quả đây là một bài tập rất tốt về việc *cùng một vấn đề có thể tấn công ở nhiều tầng trừu tượng*, và ba lời giải không loại trừ nhau: người ta dùng FlashAttention *bên trong* Swin.

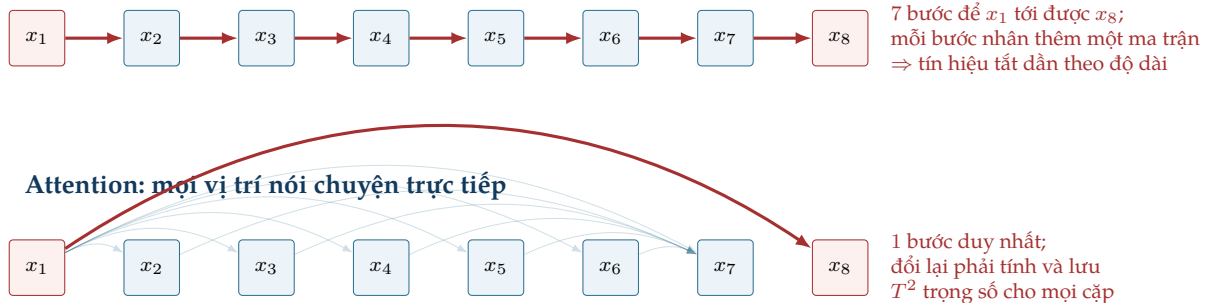
6 Vấn đề cả ba hướng trên đều vá chứ không xoá được bậc hai.

Giải pháp **mô hình không gian trạng thái** (*state space model, SSM*) như Mamba [17] quay lại dạng hồi quy tuyến tính, với tham số phụ thuộc đầu vào. Nó tính được bằng phép quét song song. Nhờ vậy chi phí tuyến tính theo độ dài mà vẫn huấn luyện song song được.

Giá phải trả đây là hướng đang lên, chưa phải kết luận: tính tới thời điểm viết, SSM chưa thay thế attention trong thị giác, và phần lớn kết quả mạnh nhất vẫn thuộc về các kiến trúc dựa trên attention.

→ cùng nguyên lý với C/13/01 — cổng của LSTM và kết nối tắt của ResNet là cùng một mẹo — tạo một đường cộng để gradient không bị nhân dồn

Mạng hồi quy: thông tin bò từng bước



Hình 1.6. Đại lượng đáng dùng để so mọi kiến trúc chuỗi: *độ dài đường đi của thông tin* giữa hai vị trí xa nhau. Ở mạng hồi quy nó bằng khoảng cách giữa hai vị trí, nên tín hiệu phải sống sót qua từng ấy phép nhân ma trận — đây chính là vanishing gradient của mục C/12/02, trải theo trục thời gian. Ở attention nó luôn bằng 1, và cái giá là các cạnh mờ trong hình: mọi cặp đều phải có một trọng số, nên số cạnh tăng theo T^2 . LSTM nằm giữa hai thái cực: đường đi vẫn dài, nhưng cổng quên tạo một đường cộng nên tín hiệu không bị nhân dồn.

Hình 1.6 là dạng hình học của ẩn dụ ở đầu mục, và nó cũng cho thấy vì sao ba bước đầu của mạch phải đi theo đúng thứ tự đó.

C. Vì sao chia cho $\sqrt{d}$ — một phép tính hai dòng

Chi tiết $\sqrt{d}$ trong công thức attention trông như một hệ số làm đẹp. Nhưng bỏ nó đi thì mô hình không huấn luyện được, và lý do tính ra được. Giả sử các thành phần của q và k độc lập, trung bình 0, phương sai 1. Tích vô hướng $q \cdot k = \sum_{i=1}^d q_i k_i$ là tổng của d số hạng độc lập, mỗi số hạng

trung bình 0 và phương sai 1. Nên nó có phương sai d , tức độ lệch chuẩn $\sqrt{d}$. Với $d = 64$, độ lệch chuẩn của các logit là 8.

Ý nghĩa của con số đó: softmax trên một tập logit có độ lệch chuẩn 8 gần như luôn dồn toàn bộ khối lượng vào một phần tử, tức nó bão hoà thành một vectơ gần one-hot. Mà đạo hàm của softmax ở vùng bão hoà xấp xỉ 0 — đúng hiện tượng đã gặp với sigmoid ở C/12/02. Chia cho $\sqrt{d}$ đưa độ lệch chuẩn về 1, giữ softmax ở vùng có gradient. Nói gọn: $\sqrt{d}$ không phải chuẩn hoá cho đẹp, nó là điều kiện để attention có gradient khi d lớn.

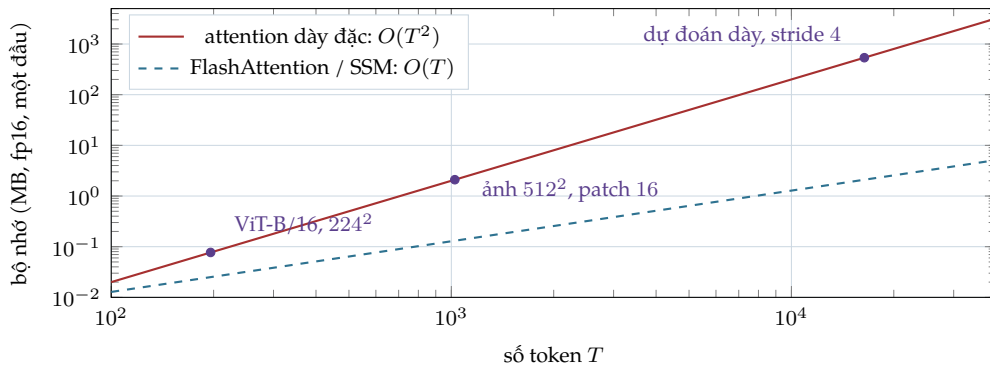
↔ **trường hợp riêng của C/12/02** — đây đúng là hiện tượng bão hoà của sigmoid, chỉ khác là xảy ra ở softmax và nguyên nhân là thang của tích vô hướng

D. Chi phí bậc hai: con số cụ thể trước, giải pháp sau

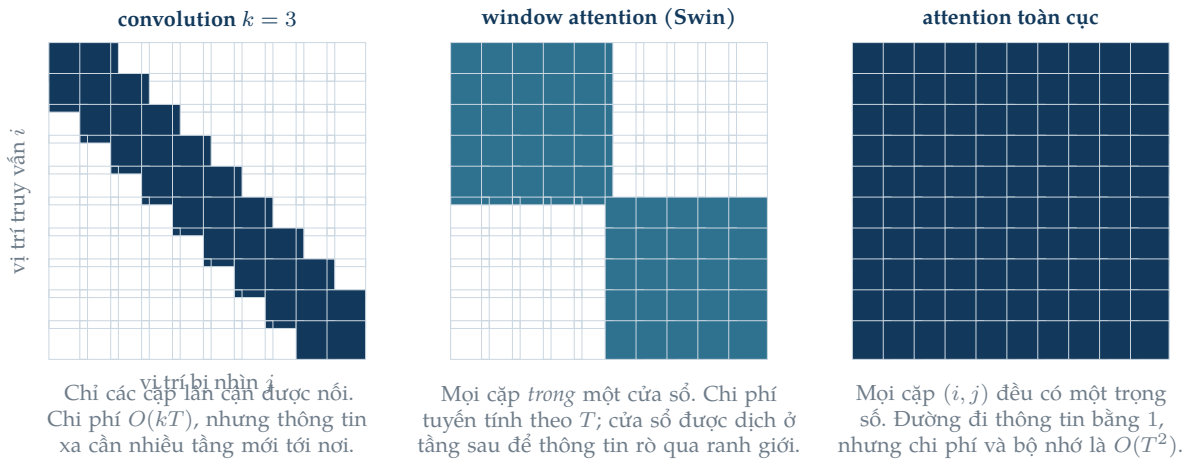
Trước khi bàn giải pháp, hãy làm phép tính để biết vấn đề nghiêm trọng tới mức nào. Ma trận attention có kích thước $T \times T$ cho mỗi đầu và mỗi tầng.

- **Phân loại ảnh, ViT-B/16 ở 224×224 :** $T = (224/16)^2 = 196$ token, nên ma trận có $196^2 = 38\,416$ phần tử — không đáng kể.
- **Ảnh 512×512 , patch 16:** $T = 1024$, ma trận có hơn một triệu phần tử. Vẫn chạy được.
- **Dự đoán dài ở stride 4 trên ảnh 512×512 :** $T = 128^2 = 16\,384$, ma trận có $2,68 \cdot 10^8$ phần tử, tức khoảng 537 MB ở fp16 — cho một đầu, một tầng. Nhân với 12 đầu là 6,4 GB chỉ để chứa các trọng số attention của một tầng.

Con số cuối là toàn bộ lý do các biến thể ở Bảng 1.3 tồn tại, và cũng là lý do ViT phẳng không dùng trực tiếp được cho segmentation. Hình 1.7 vẽ cùng ba mốc đó trên trục log để thấy chỗ đường cong bẻ gãy ngân sách.



Hình 1.7. Bộ nhớ cần để chứa ma trận attention theo số token, thang log hai chiều. Ba chấm là ba chế độ dùng thật. Khoảng cách giữa hai đường là lý do tồn tại của cả ba hướng tấn công ở Bảng 1.3; lưu ý FlashAttention nằm trên đường tuyến tính về bộ nhớ nhưng vẫn làm đủ $O(T^2)$ phép tính.



Hình 1.8. Ba kiến trúc, cùng một cách nhìn: ô (i, j) được tô nghĩa là vị trí i được phép lấy thông tin trực tiếp từ vị trí j . Convolution là một dải hẹp quanh đường chéo — đó chính là “tính cục bộ” phát biểu dưới dạng ma trận. Window attention là các khối trên đường chéo, tức một thoả hiệp: dày đặc trong cửa sổ, rỗng ngoài cửa sổ. Attention toàn cục tô kín, và diện tích phần tô chính là chi phí phải trả. Nhìn ba lưới cạnh nhau thì “inductive bias” thôi không còn là một khái niệm trừu tượng: nó chính là các ô *bị bỏ trống* vì kiến trúc quyết định trước rằng chúng không đáng quan tâm.

Hình 1.8 nên được xem trước khi đọc Bảng 1.3: hai trong ba hướng tấn công ở đó chính là hai cách khác nhau để *bớt tô* lưới bên phải, còn hướng thứ ba giữ nguyên lưới nhưng đổi cách nó được tính.

Bảng 1.3. Ba hướng tấn công chi phí bậc hai, ở ba tầng trừu tượng khác nhau. Chúng không loại trừ nhau và trên thực tế hay được dùng chồng lên nhau.

Tầng tấn công	Đại diện	Cơ chế	Mất gì / hỏng khi nào
Toán học	Linear attention	Đổi thứ tự nhân ma trận nhờ một xấp xỉ của softmax, biến $O(T^2d)$ thành $O(Td^2)$	Chất lượng giảm ở tác vụ cần attention “sắc”, tức cần chọn đúng một vị trí trong nhiều vị trí giống nhau
Kiến trúc	Swin, window attention [18]	Chỉ cho attention trong cửa sổ cố định, rồi dịch cửa sổ giữa các tầng để thông tin rò qua	Quan hệ xa cần nhiều tầng mới nối được; xuất hiện lại một dạng “đường đi dài” mà attention sinh ra để xoá
Hệ thống	FlashAttention [19]	Chia ô và tính softmax trực tuyến để không bao giờ dựng ma trận $T \times T$ trong HBM	Không giảm FLOPs và không giảm độ phức tạp tiệm cận; lợi ích phụ thuộc kích thước SRAM của GPU cụ thể

Bảng 1.3 đáng đọc kỹ vì nó dạy một thói quen chung hơn cả nội dung: khi gặp một rào cản, hãy hỏi nó có thể bị tấn công ở tầng nào. FlashAttention là ví dụ sạch nhất — nó không sửa thuật toán, nó sửa *cách thuật toán chạm vào bộ nhớ*, và với người đã tối ưu kernel CUDA thì đây là câu chuyện quen thuộc: tăng cường độ số học bằng cách giữ dữ liệu trên chip và giảm số vòng đọc ghi ra bộ nhớ ngoài.

↔ **ràng buộc bởi** B/11 — FlashAttention chỉ có ý nghĩa trong mô hình roofline: nó dời kernel từ phía ngiên bằng thông sang phía ngiên tính toán

E. Giới hạn và trường hợp hỏng

Attention không phải luôn thắng, và ba chỗ nó thua đáng nhớ.

Thứ nhất, với chuỗi ngắn và dữ liệu ít, mạng hồi quy vẫn cạnh tranh. Chi phí bậc hai chỉ là vấn đề khi T lớn; khi T nhỏ, phần chi phí thật nằm ở mạng truyền thẳng chứ không ở attention, còn transformer thì lại cần nhiều dữ liệu hơn vì nó cài ít giả định hơn.

Thứ hai, mã hoá vị trí là chỗ rò rỉ giả định. Mọi phép ngoại suy ra độ dài chưa từng thấy khi huấn luyện đều là ngoại suy, và mô hình có thể xuống cấp lạng lẽ. Với ảnh, đổi độ phân giải đầu vào mà không nội suy lại positional embedding là một lỗi phổ biến, cho ra kết quả tệ mà không báo lỗi.

Thứ ba, linear attention và window attention đánh đổi ở hai chỗ khác nhau, nên đừng coi chúng thay thế được nhau. Linear attention mất khả năng chọn sắc; window attention giữ được độ sắc trong cửa sổ nhưng mất quan hệ xa. Nếu tác vụ của bạn cần cả hai — ví dụ theo vết một đối tượng nhỏ qua toàn ảnh — thì cả hai đều không đủ và bạn phải quay lại kiến trúc phân cấp.

Về mức độ tin cậy: các con số bộ nhớ ở mục D là số học thuần túy nên chắc chắn. Còn phát biểu “SSM chưa thay thế attention trong thị giác” chỉ là đánh giá của tôi tính tới tháng 9/2026. Nó dựa trên việc các kết quả dẫn đầu vẫn dùng attention, và có thể sai trong vòng một năm.

CẠM BẤY

Cạm bẫy thực hành hay gặp nhất khi mang một ViT sang bài toán của mình: đổi kích thước ảnh đầu vào mà quên nội suy positional embedding. Số token thay đổi, nhưng bảng vị trí đã học vẫn có kích thước cũ; tùy cài đặt mà bạn nhận một exception — trường hợp may mắn — hoặc một phép cắt/đệm âm thầm cho ra một mô hình hoạt động nhưng kém hẳn. Dấu hiệu nhận biết: điểm số tụt đúng một bậc ngay khi đổi độ phân giải, trong khi mọi thứ khác giữ nguyên.

CẦN NHỚ

Đo mọi kiến trúc chuỗi bằng ba câu: thông tin đi từ i tới j qua bao nhiêu bước, chi phí tăng theo T như thế nào, và cái gì đang giữ vai trò “biết thứ tự”. RNN cho đường đi $O(T)$ và chi phí $O(T)$; attention cho đường đi $O(1)$ và chi phí $O(T^2)$; mọi biến thể là một điểm nằm giữa hai thái cực đó. [T2]

F. Kết nối

Bước hai của mạch — cổng tạo đường cộng — là cùng một ý tưởng với kết nối tắt ở [C/13/01-dong-cnn](#), và cả hai đều là hệ quả của lập luận tích Jacobian ở [C/12/02-lam-cho-training-chay-duoc](#); nếu chỉ nhớ một liên kết trong cả chương 13 thì nên nhớ liên kết này. Mục tiếp theo, [C/13/03-vit-va-inductive-bias](#), mang attention vào ảnh và hỏi cái giá của việc vứt bỏ tính cục bộ. Phần FlashAttention nối trực tiếp sang [B/11-gioi-han-tinh-toan](#) (cường độ số học, roofline) và sang [D/20-toi-uu-va-nen-mo-hinh](#). Còn attention với vai trò cơ chế lựa chọn trong bài toán thị giác cụ thể xuất hiện lại ở [C/14-bai-toan-cv-loi](#) qua DETR [20].

■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao vanishing gradient trong RNN tệ hơn trong một mạng truyền thẳng cùng độ sâu, dù cơ chế là như nhau?
2. Cổng của LSTM và kết nối tắt của ResNet giống nhau ở điểm cốt lõi nào, và khác nhau ở điểm nào?
3. Nếu bỏ hệ số $1/\sqrt{d}$ trong attention thì hỏng ở đâu, và vì sao hậu quả nặng hơn khi d lớn?

4. FlashAttention không giảm FLOPs — vậy nó nhanh hơn nhờ đâu, và trên phần cứng nào lợi ích của nó nhỏ đi?
5. Ba hướng tấn công chi phí bậc hai nằm ở ba tầng khác nhau; với bài toán segmentation ảnh 1024×1024 thì bạn chọn hướng nào trước, và vì sao?
6. Bạn fine-tune một ViT ở độ phân giải gấp đôi lúc pretrain và kết quả tệ đi. Nghi phạm số một là gì?

Nguồn

- [1] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [3] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [4] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [5] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [6] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [7] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [8] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [9] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [10] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [11] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [12] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [13] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [14] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [15] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [16] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [17] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [18] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [19] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [20] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [21] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [22] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.

- [23] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.

1.4 ViT và bài học về inductive bias (Vision Transformers and Inductive Bias)

PHỤ THUỘC C/13/01-dong-cnn (ba giả định mà convolution cài sẵn); C/13/02-tu-chuoi-toi-attention (attention, mã hoá vị trí, chỉ phí bậc hai).

TÓM TẮT

Mục này dùng *Vision Transformer* làm thí nghiệm sạch nhất cho một câu hỏi trung tâm của cả cây: cài sẵn giả định vào mô hình thì được gì và mất gì. ViT vứt bỏ hai giả định mà convolution cài sẵn, và cái giá lộ ra rất rõ ràng dưới dạng *lượng dữ liệu cần có*. Đọc xong bạn quyết định được khi nào nên dùng ViT, khi nào nên dùng CNN, khi nào nên lai. Bạn cũng đọc được mã nguồn của một hệ detection trong mười phút, nhờ cách tách *backbone*, *neck* và *head*. Nếu chỉ nhớ một điều: *inductive bias là một khoản vay trả trước cho dữ liệu* — bạn hoặc trả bằng giả định, hoặc trả bằng mẫu, và ViT chỉ đơn giản chọn cách trả thứ hai.

KIẾN THỨC CẦN CÓ TRƯỚC

- **inductive bias** — tập giả định mà mô hình mang sẵn trước khi nhìn dữ liệu, ví dụ “pixel gần nhau thì liên quan”; nó thu hẹp không gian giả thuyết nên học nhanh hơn với ít dữ liệu, nhưng nếu giả định sai thì nó chặn luôn nghiệm đúng.
- **patch** — ô vuông nhỏ được cắt ra từ ảnh, thường 16×16 pixel, rồi làm phẳng thành một vector để coi như một token; đây là cách ViT biến ảnh thành chuỗi.
- **đẳng biến và bất biến (equivariance vs invariance)** — đẳng biến tịnh tiến nghĩa là dịch ảnh thì bản đồ đặc trưng dịch theo đúng như vậy; bất biến nghĩa là dịch ảnh thì đầu ra *không đổi*. Convolution cho đẳng biến, còn phép gộp toàn cục ở cuối mới tạo ra bất biến.
- **pretraining và fine-tuning** — huấn luyện trước trên một tập rất lớn để học đặc trưng chung, rồi huấn luyện tiếp trên tập nhỏ của bài toán đích; đây là cách phổ biến nhất để bù lượng dữ liệu mà một mô hình ít giả định đòi hỏi.
- **chưng cất (distillation)** — huấn luyện mô hình học sinh bắt chước đầu ra của một mô hình thầy đã tốt, thay vì chỉ học từ nhãn cứng; nó chuyển được cả các giả định ngầm của thầy sang trò.
- **backbone, neck, head** — ba tầng chức năng của một hệ thị giác: backbone trích đặc trưng từ ảnh, neck hợp nhất đặc trưng ở nhiều tỉ lệ, head sinh đầu ra theo đúng bài toán (nhân lớp, hộp, mặt nạ).
- **FPN (feature pyramid network)** — một kiểu neck ghép đặc trưng độ phân giải cao nhưng ngữ nghĩa yếu ở tầng nông với đặc trưng ngữ nghĩa mạnh nhưng thô ở tầng sâu, để phát hiện được cả vật nhỏ lẫn vật lớn.

A. Đặt vấn đề

Suốt chương 13 ta lặp lại một câu hỏi: nên cài sẵn bao nhiêu giả định vào mô hình? Câu hỏi đó rất khó trả lời bằng lý luận, vì hai vế của nó nằm ở hai đơn vị khác nhau. Một bên là “mô hình biết trước điều gì”; bên kia là “có bao nhiêu dữ liệu”. *Vision Transformer* (ViT) [21] biến câu hỏi đó thành một thí nghiệm đo được: lấy một kiến trúc gần như không có giả định thị giác nào, huấn luyện nó ở nhiều quy mô dữ liệu khác nhau, và xem điểm giao cắt nằm ở đâu.

Kết quả là câu trả lời sạch nhất mà ngành có: với dữ liệu ít, ViT thua CNN rõ rệt; với dữ liệu rất nhiều, ViT vượt. Giá trị cụ thể của mục này là biến quan sát đó thành một quy tắc quyết định dùng được. Quy tắc ấy chạy trên ba dữ kiện: bạn có bao nhiêu ảnh có nhãn, bạn có mô hình

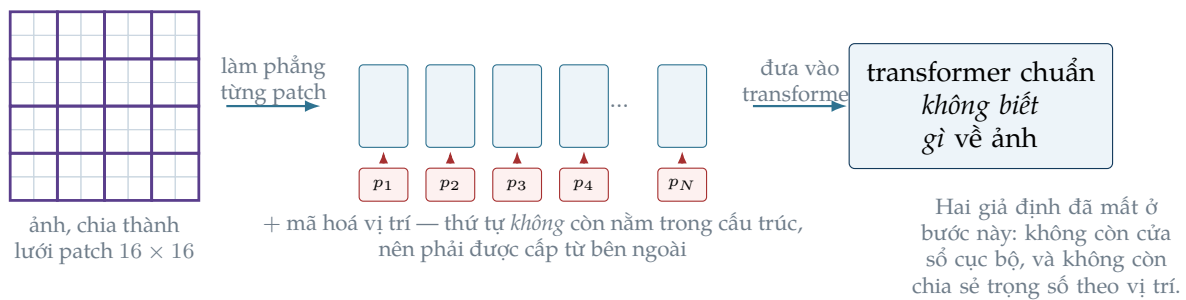
pretrain nào, và ràng buộc độ phân giải của bạn là gì. Mục này cũng cho bạn một cách đọc mã nguồn nhanh cho bất kỳ hệ thị giác nào.

→ **tiền đề** B/10 — trực đánh đổi giả định—dữ liệu được phát biểu ở chương thiết kế kiến trúc; mục này là phép đo thực nghiệm cho chính trực đó

B. ViT bỏ đi cái gì

ViT cắt ảnh thành các patch 16×16 , làm phẳng mỗi patch thành một vector, cộng mã hoá vị trí, rồi đưa cả dãy vào một transformer chuẩn. Phép biến đổi đó vứt bỏ hai trong ba giả định của convolution nêu ở C/13/01.

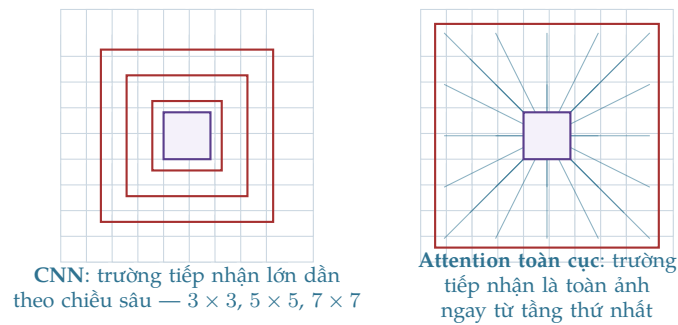
- **Tính cục bộ biến mất.** Ngay từ tầng đầu tiên, mọi patch nói chuyện với mọi patch. Không có gì trong kiến trúc nói rằng hai patch cạnh nhau liên quan nhiều hơn hai patch ở hai góc ảnh — mô hình phải học điều đó từ dữ liệu.
- **Đẳng biến tịnh tiến biến mất.** Convolution dùng chung một bộ trọng số cho mọi vị trí nên dịch ảnh thì đặc trưng dịch theo. ViT gán một mã vị trí riêng cho từng ô, nên cùng một vật ở hai vị trí khác nhau đi qua hai đường tính toán hơi khác nhau.
- **Phân cấp cũng gần như biến mất** ở bản ViT gốc: mọi tầng chạy ở cùng một độ phân giải token, không có chuỗi giảm dần độ phân giải như CNN. Đây là giả định thứ ba, và chính nó sẽ được các biến thể phân cấp lấy lại đầu tiên.



Hình 1.9. Ba bước biến một ảnh thành thứ mà transformer xử lý được, và chỗ inductive bias rơi ra. Việc cắt patch vẫn giữ lại một chút tính cục bộ: 256 điểm ảnh trong một ô bị gộp cứng. Nhưng *quan hệ giữa* các patch thì không còn được mã hoá ở đâu trong kiến trúc. Nó phải được cấp bằng một bảng vị trí học được. Đó là lý do đổi độ phân giải đầu vào mà quên nội suy bảng này làm mô hình xuống cấp lạng lẽ: bạn vừa cấp sai đúng cái thông tin mà kiến trúc không tự có.

Hình 1.9 cho thấy điều mà câu “ViT coi ảnh như một chuỗi” còn giấu: chuỗi đó không có thứ tự tự nhiên, và toàn bộ hiểu biết về không gian hai chiều bị dồn vào một bảng tham số phụ.

Cần nói ngay một điều để tránh hiểu quá đà: *ViT không phải không có giả định*. Bản thân việc cắt thành patch là một giả định cục bộ khá mạnh — 256 pixel trong một ô bị gộp thành một token và không bao giờ được xét riêng nữa. Ngoài ra, các phân tích attention cho thấy ở những tầng đầu, nhiều đầu tự học ra hành vi gần như cục bộ, tức mô hình khám phá lại một phần giả định đã bị vứt. Phát biểu đúng là: ViT giảm mạnh lượng giả định cài sẵn, chứ không xoá sạch.

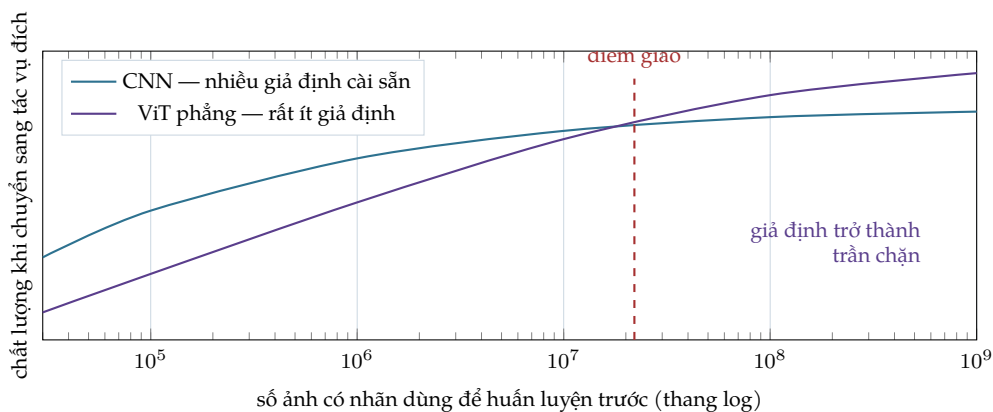


Hình 1.10. Hai cách phủ sóng thông tin. CNN mở rộng trường tiếp nhận dần dần và do đó *cài sẵn* rằng quan hệ gần quan trọng hơn quan hệ xa; attention toàn cục nối mọi vị trí ngay lập tức và không cài giả định nào về khoảng cách, đổi lại phải học quan hệ đó từ dữ liệu và trả chi phí bậc hai. Swin và các biến thể cửa sổ nằm giữa hai thái cực này.

Hình 1.10 là cách gọn nhất để thấy cả ưu điểm lẫn cái giá: bên trái, thông tin xa cần nhiều tầng nhưng mỗi tầng rẻ và có sẵn tiên nghiệm đúng cho ảnh tự nhiên; bên phải, thông tin xa có ngay nhưng phải trả bằng T^2 cặp quan hệ và bằng việc mô hình không được cho biết cặp nào đáng quan tâm.

C. Cái giá hiện ra dưới dạng lượng dữ liệu

Kết quả gốc của ViT rất dễ tóm tắt: huấn luyện từ đầu trên ImageNet-1k thì nó thua các ResNet cùng hạng; pretrain trên một tập lớn hơn nhiều bậc rồi fine-tune thì nó vượt [21]. Đây là minh chứng trực tiếp cho trục đánh đổi giả định–dữ liệu: giả định mà bạn không cài vào kiến trúc thì phải mua bằng mẫu.



Hình 1.11. Trục đánh đổi giả định–dữ liệu, vẽ định tính. Ở phía trái, các giả định mà convolution cài sẵn là món quà miễn phí: chúng thu hẹp không gian giả thuyết đúng chỗ, nên CNN học được từ ít mẫu. Ở phía phải, chính các giả định ấy thành trần — chúng cấm mô hình biểu diễn những quan hệ mà dữ liệu đã đủ để dạy nó. Điều quan trọng nhất trong hình là *có* một điểm giao chứ không phải vị trí của nó: vị trí phụ thuộc tác vụ, recipe và quy mô mô hình, và ba cách trả nợ giả định ở mục C đều là cách kéo đường tím sang trái.

Hình 1.11 biến một câu định tính thành một hình có thể tranh luận được: câu hỏi “CNN hay ViT” thực chất là câu hỏi “tôi đang đứng ở đâu trên trục hoành”.

Điều làm câu chuyện thú vị là có ít nhất ba cách trả khoản nợ đó, và chúng trả bằng ba loại tiền khác nhau.

- **Trả bằng dữ liệu** — pretrain trên tập cực lớn, rồi fine-tune. Hiệu quả nhất về chất lượng, nhưng chỉ khả thi nếu bạn có tập đó hoặc có checkpoint của người khác; và nó đẩy toàn bộ chi phí sang một pha huấn luyện mà bạn không kiểm soát.
- **Trả bằng chính quy hoá và chứng cất** — DeiT cho thấy với augmentation mạnh, lịch học dài và một thầy CNN để chứng cất, ViT huấn luyện được trên riêng ImageNet-1k [22]. Đáng chú ý là thầy CNN chuyển sang trò không chỉ nhận mà cả *giả định* của mình — đây là một cách cài inductive bias qua cửa sau.
- **Trả bằng kiến trúc** — lắp lại một phần giả định đã bỏ, tức các biến thể phân cấp ở mục D. Rẻ nhất về dữ liệu, nhưng đánh mất sự thuần khiết và một phần khả năng mở rộng theo quy mô.

↪ **hệ quả của C/18** — pretraining tự giám sát là cách rẻ nhất để trả khoản nợ giả định; các backbone mạnh nhất hiện nay đều đến từ hướng này.

↪ **cùng nguyên lý với C/12/04** — augmentation mạnh và chứng cất đều bơm thêm giả định vào mô hình, chỉ khác đường đi.

D. Phản ứng thứ nhất: ViT phân cấp

ViT phẳng có hai vấn đề thực dụng ngoài chuyện dữ liệu: nó chỉ sinh ra đặc trưng ở một tỉ lệ duy nhất, trong khi detection và segmentation cần nhiều tỉ lệ; và chi phí bậc hai chặn đường tới độ phân giải cao, như đã tính ở C/13/02. Swin giải cả hai bằng một thay đổi: giới hạn attention trong các cửa sổ cố định rồi *dịch* cửa sổ ở tầng kế tiếp để thông tin rò qua ranh giới, đồng thời gộp patch dần để tạo kim tự tháp đặc trưng [18]. Chi phí trở thành tuyến tính theo số token, và đầu ra có đúng dạng mà FPN cần. PVT đi theo hướng gần giống nhưng giảm số khoá và giá trị thay vì giới hạn cửa sổ. Còn lại ghép đơn giản nhất là thay phép cắt patch bằng vài tầng convolution làm “stem”. Cách này được biết là làm giai đoạn huấn luyện đầu ổn định hơn hẳn. Đây là kinh nghiệm thực hành rất phổ biến, không phải một kết quả tôi kiểm chứng lại.

Cái giá thì cụ thể: cửa sổ tạo lại một dạng “đường đi dài” cho quan hệ xa, đúng thứ mà attention sinh ra để xoá; và kiến trúc nhiều giai đoạn khó mở rộng theo quy mô một cách trơn tru hơn so với một chồng khối đồng nhất.

E. Phản ứng thứ hai: ConvNeXt, và vì sao nó là phản chứng quan trọng

Hướng ngược lại không sửa transformer mà sửa cách so sánh. ConvNeXt giữ nguyên convolution rồi lần lượt áp từng thành phần recipe của transformer, đo sau mỗi bước, và kết quả là CNN bắt kịp [11]. Kết luận đã nêu ở C/13/01 và nhắc lại ở đây vì nó thay đổi cách đọc chính mục này: phần lớn khoảng cách được gán cho “attention thay convolution” thực ra thuộc về công thức huấn luyện. Nói cách khác, thí nghiệm ViT vẫn đúng về trục giả định–dữ liệu, nhưng *độ lớn* của hiệu ứng kiến trúc đã bị phóng đại trong cách trình bày đương thời.

F. Ba dạng tổ chức và ba lớp bài toán

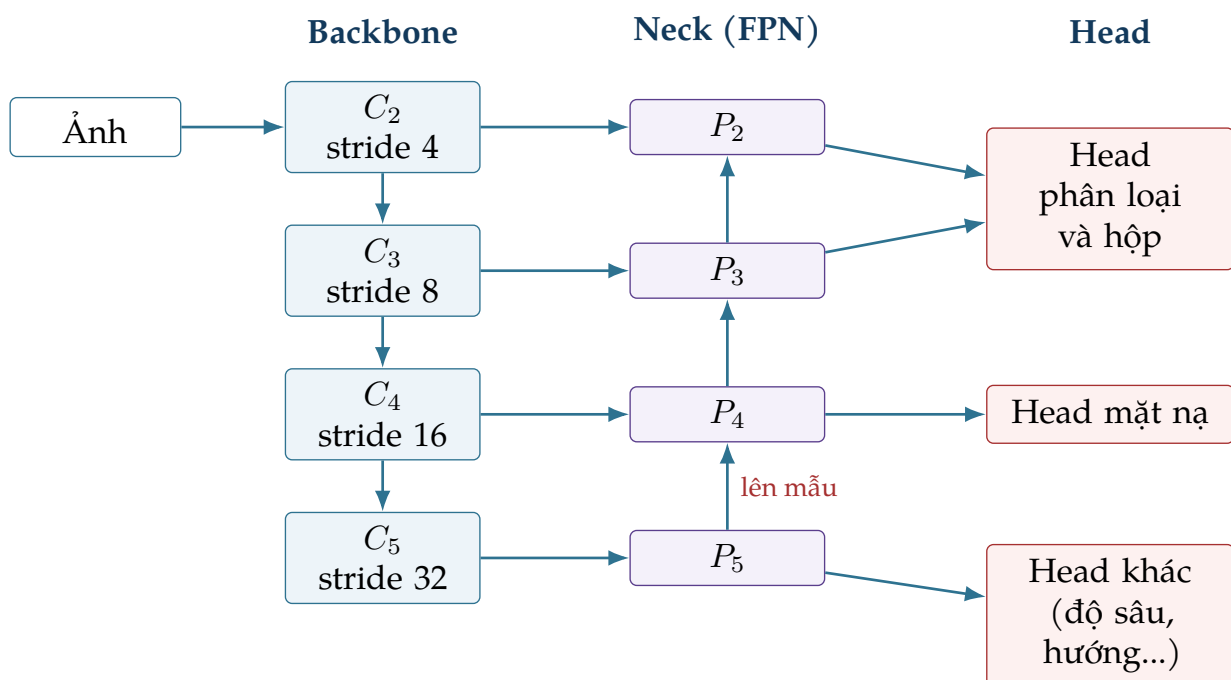
Chọn sai dạng tổ chức là lỗi thiết kế cơ bản, không phải lỗi tinh chỉnh. Không có siêu tham số nào cứu được một encoder-only đang bị bắt sinh chuỗi tự hồi quy.

Bảng 1.4. Ba dạng tổ chức của một mô hình dựa trên transformer, và lớp bài toán tương ứng.

Dạng	Cơ chế	Hợp với bài toán nào	Hỏng khi nào
Encoder-only	Mọi token thấy mọi token, một lượt	Hiểu và dự đoán từ toàn bộ đầu vào: phân loại, dự đoán đầy, trích đặc trưng cho hệ khác	Cần sinh chuỗi có độ dài chưa biết trước, hoặc cần sinh tự hồi quy
Decoder-only	Attention có mặt nạ nhân quả, sinh từng token dựa trên các token đã sinh	Sinh tự hồi quy: ngôn ngữ, chuỗi hành động, mã hoá ảnh thành token rồi sinh	Suy luận chậm vì tuần tự; lãng phí khi đầu ra là một nhãn duy nhất
Encoder-decoder	Encoder đọc đầu vào, decoder sinh đầu ra và tra cứu encoder bằng cross-attention	Biến đổi chuỗi sang chuỗi; trong thị giác là DETR với tập truy vấn học được [20]	Nhiều tham số và nhiều đường nối hơn; hội tụ chậm hơn ở bài toán nhỏ

G. Tách backbone, neck, head — cách đọc code nhanh nhất

Ba khái niệm này không phải lý thuyết mà là quy ước kỹ thuật, và chúng có giá trị thực dụng rất cao: gần như mọi repo detection và segmentation hiện đại đều tổ chức theo đúng ba khối này, nên nắm được chúng là đọc được cấu trúc một repo lạ trong mười phút thay vì một ngày.



Hình 1.12. Ba khối chức năng của một hệ thị giác đầy. Backbone sinh đặc trưng ở nhiều stride; neck hợp nhất chúng theo chiều từ sâu về nông để mọi tỉ lệ đều có ngữ nghĩa mạnh; head chuyển đặc trưng thành đầu ra của bài toán. Thay backbone là thao tác rẻ nhất và hay làm nhất; thay neck ảnh hưởng chủ yếu tới vật nhỏ; thay head là đổi bài toán.

Hình 1.12 cũng giải thích vì sao ViT phẳng khó lắp vào hệ detection: nó chỉ sinh ra một cột duy nhất chứ không sinh ra bốn mức C_2 tới C_5 , nên hoặc phải chế thêm một neck đặc biệt, hoặc phải chuyển sang biến thể phân cấp. Đây là một ràng buộc kỹ thuật rất thực tế và thường quyết định lựa chọn backbone hơn cả điểm số ImageNet.

→ **tiền đề** C/14 — các head của detection và segmentation giả định có sẵn bốn mức đặc trưng, nên dạng đầu ra của backbone quyết định head nào lắp được

H. Giới hạn và trường hợp hỏng

Bốn chỗ hỏng đáng nhớ, xếp theo tần suất gặp trong thực hành.

Thứ nhất, *ViT huấn luyện từ đầu trên tập nhỏ*. Đây là cách phổ biến nhất để kết luận sai rằng “transformer không hợp bài toán của tôi”. Nếu bạn có vài nghìn ảnh, kết luận đúng là bạn không đủ dữ liệu để trả khoản nợ giả định, chứ không phải kiến trúc kém.

Thứ hai, *kích thước patch so với kích thước đối tượng*. Một patch 16×16 nghiền mọi đối tượng nhỏ hơn 16 pixel thành một phần của một token duy nhất. Với ảnh camera gắn xe, người đi bộ ở xa chỉ chiếm mười pixel. Đây không phải chi tiết nhỏ mà là giới hạn cứng. Nó giải thích vì sao các hệ dự đoán dày dùm patch nhỏ hoặc dùng conv stem.

Thứ ba, *mã hoá vị trí khi đổi độ phân giải*, đã nói ở mục trước và nhắc lại vì nó thực sự hay xảy ra.

Thứ tư, *mất đẳng biến tịnh tiến là có thật và đo được*. Cùng một vật ở hai vị trí có thể cho đặc trưng hơi khác nhau. Trong phần lớn trường hợp, augmentation ngẫu nhiên đã bù đủ. Nhưng ở các bài toán đo lường chính xác như ước lượng tư thế hay đo khoảng cách, đây là một nguồn sai số hệ thống đáng kiểm tra chứ không nên giả định là không có.

Về độ tin cậy: quan hệ “ít dữ liệu thì CNN thắng, nhiều dữ liệu thì ViT thắng” là kết quả đã công bố và được lặp lại rộng rãi; còn mức dữ liệu cụ thể ở điểm giao cắt phụ thuộc tác vụ, recipe và quy mô mô hình tới mức tôi không nêu một con số nào ở đây.

CẠM BẮY

Cạm bẫy tư duy lớn nhất của mục này là đọc “ViT không có inductive bias” theo nghĩa đen rồi suy ra “ViT là mô hình trung lập, không giả định gì”. Không đúng. Phép cắt patch là một giả định cục bộ rất mạnh. Mã hoá vị trí là một giả định về cách biểu diễn không gian. Và cách chọn dữ liệu pretrain là một giả định khổng lồ về thế giới mà mô hình sẽ gặp. Phát biểu chính xác là ViT *dịch chuyển* giả định từ kiến trúc sang dữ liệu và recipe — và giả định nằm trong dữ liệu thì khó nhìn thấy hơn nhiều so với giả định nằm trong kiến trúc.

CẦN NHỚ

Inductive bias là khoản vay trả trước cho dữ liệu: giả định bạn không cài vào kiến trúc thì phải mua bằng mẫu, bằng chưng cất, hoặc bằng augmentation. Khi chọn giữa CNN, ViT và các biến thể lai, câu hỏi đầu tiên không phải “cái nào mạnh hơn” mà là “tôi có bao nhiêu dữ liệu và bao nhiêu tiền nghiệm đáng tin về bài toán này”. [T2]

I. Kết nối

Mục này khép lại chương 13 bằng cách trả lời câu hỏi mở ở [C/13/01-dong-cnn](#): ba giả định của convolution đáng giá bao nhiêu, đo bằng lượng dữ liệu cần để thay thế chúng. Nó dựa hoàn toàn vào cơ chế attention ở [C/13/02-tu-chuoi-toi-attention](#), và cách tách backbone-neck-head ở mục G là tiền đề trực tiếp cho [C/14-bai-toan-cv-loi](#), nơi các head cụ thể được thiết kế. Về phía dữ liệu, việc trả nợ giả định bằng pretraining là nội dung của [C/18-pretraining-va-foundation](#), còn việc trả bằng augmentation đã nằm ở [C/12/04-tong-quat-hoa-va-chan-doan](#). Cuối cùng, bài học ConvNeXt nổi sang [E/24-thi-nghiem-va-ablation](#), chương duy nhất trong cây dạy cách không bị một bảng so sánh đánh lừa.

Câu hỏi tự kiểm (trả lời không nhìn bài)

- ViT bỏ đi hai giả định nào của convolution, và mỗi giả định bị bỏ gây ra hậu quả đo được nào?

2. Vì sao nói ViT vẫn có inductive bias, dù nó bỏ tính cục bộ và đẳng biến tịnh tiến?
3. Ba cách bù lại lượng dữ liệu mà ViT đòi hỏi là gì, và mỗi cách trả giá bằng loại tài nguyên nào?
4. Vì sao Swin lắp vào một hệ detection dễ hơn ViT phẳng — câu trả lời nằm ở khối nào trong sơ đồ backbone–neck–head?
5. Nếu ConvNeXt cho thấy recipe quan trọng hơn kiến trúc, điều đó thay đổi cách bạn đọc một bảng so sánh backbone thế nào?
6. Bạn cần phát hiện người đi bộ cách xa 80 mét, chỉ chiếm khoảng 12 pixel chiều cao. Vì sao ViT patch 16 là lựa chọn tệ, và hai cách sửa là gì?
7. Trong hệ của bạn, thay backbone, thay neck và thay head có tác động khác nhau thế nào — và vì sao thứ tự thứ nên là backbone trước?

Nguồn

- [1] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [3] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [4] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [5] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [6] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [7] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [8] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [9] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [10] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [11] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [12] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [13] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [14] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [15] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [16] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [17] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [18] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [19] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [20] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [21] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [22] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.

- [23] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.

1.5 Thực hành

Chương này dễ đọc thụ động và khó nhớ, vì phần lớn nội dung là tên riêng. Hai mục dưới đây là phần đọc và phần tự vấn; phần chuyển nó thành thứ dùng được nằm ở khối mini project cuối chương, nơi bạn tự tạo ra bảng đo của chính mình thay vì tin bảng của người khác.

Repo và SOTA (chốt tháng 9/2026). Backbone dùng thật: [facebookresearch/dinov3](#) phủ dải từ khoảng 21 triệu tới vài tỉ tham số nhưng có giấy phép riêng, nên nếu cần điều khoản thoát thì DINOv2 (Apache 2.0) là lựa chọn thay thế [23]; ConvNeXt-V2 và Swin lấy qua [timm](#); ngoài ra EVA-02 và SigLIP. Cài đặt tối giản để *đọc* chứ không để chạy production: [lucidrains/vit-pytorch](#) và [karpathy/nanoGPT](#). Attention nhanh: [Dao-AILab/flash-attention](#). Mô hình không gian trạng thái: [state-spaces/mamba](#). Danh sách tên mô hình này chốt tại tháng 9/2026 và sẽ cũ trong sáu tới mười hai tháng; ngược lại, các repo hạ tầng như [timm](#) hay COLMAP bền hơn nhiều, nên hãy phân bổ thời gian học theo đúng tỉ lệ đó.

Câu hỏi kiểm tra hiểu. Vì sao ViT cần nhiều dữ liệu hơn CNN, và ba cách bù lại — pretraining, augmentation mạnh, lai ghép — trả giá bằng loại tài nguyên nào? FlashAttention không giảm FLOPs, vậy nó nhanh hơn nhờ đâu, và trên phần cứng nào lợi ích của nó thu hẹp lại? Nếu ConvNeXt cho thấy recipe quan trọng hơn kiến trúc, điều đó thay đổi thứ tự bạn đọc một paper kiến trúc như thế nào?

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Ba giả định của convolution	Tính cục bộ: chỉ pixel lân cận liên quan trực tiếp. Chia sẻ trọng số, tức đẳng biến tịnh tiến. Và phân cấp: đặc trưng phức tạp ghép từ đặc trưng đơn giản. Cả chương là chuyện thêm hoặc bớt ba giả định này.
•• Trường tiếp nhận	Vùng pixel đầu vào ảnh hưởng tới một giá trị đầu ra. Trong CNN nó lớn dần theo chiều sâu; trong attention toàn cục nó bằng cả ảnh ngay từ tầng đầu.
•• Vì sao xếp chồng hai tầng 3×3 thay một tầng 5×5 ?	Cùng trường tiếp nhận, nhưng tốn $18C^2$ tham số thay vì $25C^2$, và có thêm một lần phi tuyến ở giữa. Một trong ít phép đổi tốt ở cả ba mặt.
•• Vì sao nút thắt 1×1 rẻ hơn gần chín lần?	Vì phần đắt nhất của conv là tích của kênh vào, kênh ra và diện tích cửa sổ. Giảm kênh xuống 64 rồi mới làm 3×3 thì k^2 chỉ nhân với một số nhỏ: 589 824 phép nhân còn 69 632.
••• Kết nối tắt	Đường cộng $y = x + F(x)$ đưa ánh xạ đồng nhất thành phương án mặc định. Jacobian thành $I + \partial F/\partial x$, nên tích qua nhiều khối luôn còn một thành phần bằng 1. Giá phải trả: phải giữ x sống tới điểm cộng, nên tốn thêm bộ nhớ.
•• Depthwise separable	Tách conv thành một bước không gian chạy riêng từng kênh, rồi một bước 1×1 trộn kênh. Chi phí còn $1/C_{\text{out}} + 1/k^2$ lần, khoảng 0,115 với $k = 3$ và $C_{\text{out}} = 256$.
••• Giảm 8,7 lần FLOPs mà độ trễ gần như không đổi — nguyên nhân?	Bước depthwise đọc mỗi phần tử để làm vôn ven chín phép nhân, nên nó nghẽn ở băng thông chứ không ở khả năng tính toán. Phân biệt với nghẽn tính toán: đo thông lượng bộ nhớ, nếu đã sát trần thì giảm FLOPs vô ích.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
• Dilated convolution	Giãn khoảng cách giữa các điểm lấy mẫu, nên trường tiếp nhận tăng mà không phải giảm độ phân giải. Sinh ra cho segmentation; hồng thành hoa văn răng lược nếu xếp chồng nhiều tầng cùng hệ số giãn.
•• Cổng của LSTM	Cơ chế cho trạng thái ô đi qua theo dạng cộng: $c_t = f_t \odot c_{t-1} + \dots$. Khi cổng quên mở gần 1, gradient chảy qua mà không bị nhân với ma trận trọng số — đúng mẹo của kết nối tắt, chỉ khác là hệ số học được.
••• Attention	Mỗi vị trí sinh truy vấn, mọi vị trí sinh khóa và giá trị; đầu ra là tổ hợp có trọng số của mọi vị trí. Đường đi thông tin bằng 1 thay vì T , đổi lại chi phí và bộ nhớ $O(T^2)$.
••• Vì sao attention chia cho $\sqrt{d}$?	Tích vô hướng d chiều có độ lệch chuẩn $\sqrt{d}$, tức 8 khi $d = 64$. Softmax trên logit lệch chuẩn 8 bão hoà thành gần one-hot, và ở vùng đó đạo hàm bằng 0.
•• Mã hoá vị trí	Thông tin vị trí được cấp từ bên ngoài, vì phép attention bất biến với hoán vị. Bốn họ: sin tuyệt đối, học được, tương đối, và RoPE. Loại tuyệt đối ngoại suy kém ra ngoài độ dài đã thấy khi huấn luyện.
•• Window attention	Chỉ cho attention trong một cửa sổ cố định, rồi dịch cửa sổ ở tầng sau để thông tin rò qua ranh giới. Chi phí thành tuyến tính theo T ; giá phải trả là quan hệ xa cần nhiều tầng mới nối được.
• FlashAttention	Chia ô và tính softmax trực tuyến để không bao giờ dựng ma trận $T \times T$ trong HBM. Số FLOPs không đổi và độ phức tạp tiệm cận cũng không đổi; cái giảm là số vòng đọc ghi bộ nhớ ngoài.
••• ViT	Cắt ảnh thành patch, coi mỗi patch là một token, rồi đưa vào transformer chuẩn. Nó vứt bỏ tính cục bộ và đẳng biến tịnh tiến, nên thua CNN khi dữ liệu ít và vượt khi dữ liệu rất nhiều.
••• Vì sao ViT cần nhiều dữ liệu hơn CNN?	Vì giả định không cài vào kiến trúc thì phải mua bằng mẫu. Ba cách trả: pretrain trên tập rất lớn, chưng cất từ thầy CNN cộng augmentation mạnh, hoặc lắp lại giả định bằng kiến trúc phân cấp.
•• Đổi độ phân giải đầu vào rồi ViT tẻ hần — nguyên nhân?	Quên nội suy bảng mã hoá vị trí: số token đổi nhưng bảng vẫn kích thước cũ. Phân biệt với thiếu fine-tune: lỗi này làm điểm tụt ngay cả khi bạn fine-tune lại đủ lâu.
•• ViT phẳng hết bộ nhớ ở dự đoán dày stride 4 — nguyên nhân?	Số token là $128^2 = 16\,384$, nên ma trận attention chiếm khoảng 537 MB cho một đầu, một tầng. Lỗi sửa là chuyển sang biến thể cửa sổ hoặc phân cấp.
••• ConvNeXt	Một CNN được hiện đại hoá bằng chính công thức huấn luyện của ViT, và nó bắt kịp ViT cùng hạng. Bài học: phần lớn chênh lệch từng được gán cho kiến trúc thực ra thuộc về recipe.
••• Backbone, neck, head	Backbone trích đặc trưng ở nhiều stride, neck hợp nhất chúng theo chiều từ sâu về nông, head sinh đầu ra của bài toán. Đây là cách đọc một repo detection nhanh nhất, và là lý do ViT phẳng khó lắp: nó chỉ sinh một mức đặc trưng.

••• phải thuộc •• nên nắm • biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
timm	Kho backbone chuẩn của ngành: ConvNeXt, Swin, EVA, ResNet hiện đại hoá — và là nơi thấy rõ nhất rằng cùng kiến trúc với recipe khác nhau cho điểm số khác nhau	Hạ tầng bên; đáng đầu tư hơn bất kỳ tên mô hình nào

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
DINOv2 và DINOv3	Backbone ViT trả khoản nợ giả định bằng pretraining tự giám sát ở quy mô rất lớn — đúng cách thứ nhất trong ba cách ở mạch 3	DINOv2 giấy phép Apache 2.0; DINOv3 giấy phép riêng, chốt 9/2026
Swin và các biến thể phân cấp	Lắp lại tính cục bộ và đa tỉ lệ mà ViT phẳng đánh mất, để sinh ra bốn mức đặc trưng mà FPN cần	Vẫn là backbone mặc định cho nhiều hệ dự đoán dày
flash-attention	Tấn công chi phí bậc hai ở tầng hệ thống: không dựng ma trận $T \times T$ trong HBM, giữ nguyên FLOPs	Đã thành mặc định trong phần lớn thư viện transformer
ConvNeXt-V2	Bằng chứng đang sống rằng convolution vẫn cạnh tranh khi được huấn luyện bằng recipe hiện đại	Đọc kèm bảng ablation của nó là cách nhanh nhất để thấy từng thành phần recipe đáng bao nhiêu

MINI PROJECT SLAM · Chọn backbone cho bộ mô tả đặc trưng dưới ngân sách độ trễ của Orin · 1–2 ngày

Mục tiêu	biến câu hỏi “backbone nào tốt hơn” thành một quyết định có bằng chứng, dưới đúng ràng buộc của bạn. Và thấy tận mắt rằng thứ hạng theo FLOPs có thể ngược thứ hạng theo độ trễ thật.
Dữ liệu	dùng lại tập cặp ghép đã gán nhãn từ mini project chương 12, nhưng thay đầu vào thủ công bằng miếng ảnh 32×32 cắt quanh mỗi góc ORB trên EuRoC. Ba backbone cùng hạng lấy từ <code>timm</code> : một CNN cổ điển (ResNet-18), một CNN hiện đại hoặc hướng thiết bị nhúng (ConvNeXt-T hoặc MobileNetV3), và một ViT phân cấp (Swin-T).
Các bước	fine-tune cả ba ở cùng ngân sách epoch và cùng công thức huấn luyện — đây là điều kiện bắt buộc, nếu không thì bạn đang đo recipe chứ không đo kiến trúc; đo trên Jetson Orin ở batch bằng 1, tức đúng chế độ mà SLAM chạy, sau khi khoá xung nhịp, lấy cả trung vị lẫn phân vị 95; lập bảng sáu cột gồm độ chính xác ghép, số tham số, FLOPs, độ trễ trung vị, độ trễ p_{95} , bộ nhớ đỉnh.
Xong khi	bạn chỉ ra được backbone nào thắng ở một ngân sách cụ thể, chẳng hạn 8 ms mỗi khung hình. Bạn giải thích thứ hạng bằng hai đại lượng: trường tiếp nhận so với kích thước miếng ảnh, và cường độ số học so với ngưỡng roofline của Orin. Và bạn chỉ ra được ít nhất một cặp trong bảng có thứ hạng FLOPs ngược thứ hạng độ trễ.
Nối với	B/11 cho mô hình roofline dùng để giải thích cột độ trễ; C/12/03 cho việc giữ công thức huấn luyện ngang nhau giữa ba lần chạy; C/14 cho bước tiếp theo là lắp backbone đã chọn vào một head thật.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- Vì cửa sổ 3×3 không phải một giới hạn mà là một *giả định*: quan hệ gần quan trọng hơn quan hệ xa, và mẫu hình có ý nghĩa như nhau ở mọi vị trí. Giả định đó đúng với ảnh tự nhiên, nên nó thay thế cho dữ liệu. Attention toàn cục không giả định gì về khoảng cách, nên phải học quan hệ đó từ mẫu và trả thêm chi phí $O(T^2)$.
- Gradient descent không tìm ra nghiệm “các tầng thừa làm ánh xạ đồng nhất”, dù nghiệm đó tồn tại. Đây là vấn đề tối ưu hoá, không phải sức chứa — và kết nối tắt sửa nó bằng cách biến ánh xạ đồng nhất thành phương án mặc định, đồng thời để lại một thành phần bằng 1 trong tích các Jacobian.
- Vì phần depthwise đọc mỗi phần tử đầu vào để làm vôn vện $k^2 = 9$ phép nhân, nên cường độ số học của nó rất thấp và kernel rơi vào phía nghèo bằng thông bộ nhớ. Giảm FLOPs của một kernel vốn đã nghèo ở bộ nhớ thì gần như không giảm thời gian.
- Cải tiến công thức huấn luyện — optimizer, lịch học, số epoch, augmentation — nhiều hơn là cải tiến kiến trúc. Điều đó không xoá bỏ tiến bộ về kiến trúc, nhưng nó nói rằng mọi bảng so sánh không kiểm soát recipe đều không tách được hai nguồn ấy, nên câu hỏi đầu tiên khi đọc bảng phải là câu hỏi về recipe.

- A5. Không có gì trong mô hình thay đổi — thứ thay đổi là *ai trả tiền cho các giả định*. Với ít dữ liệu, giả định cài sẵn của convolution là món quà miễn phí; với rất nhiều dữ liệu, chính giả định đó thành trần, còn mô hình ít giả định thì học ra được các quan hệ mà convolution cấm nó biểu diễn.

NÓI LẠI THẬT ĐƠN GIẢN

Giả sử bạn thuê một người thợ mới và giao việc phân loại đồ vật qua ảnh. Bạn có hai cách. Cách thứ nhất là dạy trước thật nhiều quy tắc: hai chỗ cạnh nhau trong ảnh thường liên quan tới nhau; một cái cốc vẫn là cái cốc dù nó nằm ở góc nào của bàn; nhìn nét nhỏ trước rồi mới ghép thành hình lớn. Cách thứ hai là không dạy gì cả, chỉ đưa cho họ thật nhiều ví dụ và để họ tự rút ra.

Cả chương là câu chuyện về việc chọn giữa hai cách đó, và phát hiện quan trọng nhất là chúng thay thế được cho nhau. Ai chỉ có vài nghìn tấm ảnh thì dạy trước nhiều quy tắc là món hời, vì quy tắc làm thay phần việc mà ví dụ đáng ra phải làm. Ai có hàng trăm triệu tấm thì chính những quy tắc ấy lại thành xiềng: chúng cấm người thợ nhận ra các mối liên hệ mà bạn không nghĩ tới khi dạy.

Có hai cái giá phải nhớ. Một là bỏ bớt quy tắc thì phải trả bằng ví dụ, hoặc bằng việc mượn một người thợ đã được luyện sẵn ở nơi khác. Hai là “rẻ hơn trên giấy” không có nghĩa là “nhanh hơn trong xương”: một cách làm cắt được tám phần chín số phép tính vẫn có thể chạy chậm y như cũ, vì thời gian thật đi vào việc lấy vật liệu ra khỏi kho chứ không vào việc gia công. Và bất ngờ cuối cùng: phần lớn khác biệt giữa hai người thợ hoá ra không nằm ở quy tắc họ được dạy, mà ở cách họ được luyện tập.

Một câu: Quy tắc dạy trước là khoản vay trả trước cho ví dụ — ít ví dụ thì nên dạy nhiều, nhiều ví dụ thì chính lời dạy thành xiềng, và phần lớn chênh lệch bạn thấy trên bảng xếp hạng lại đến từ cách luyện tập chứ không từ lời dạy.

1.6 Kết nối

Chương này nhận đầu vào từ C/12-co-che-huan-luyen: không có bộ máy huấn luyện thì các kiến trúc ở đây chỉ là sơ đồ khối. Nó cũng nhận từ B/10-thiet-ke-kien-truc, nơi trực đánh đổi giả định-dữ liệu được phát biểu lần đầu. Xuôi xuống, nó cấp backbone cho C/14-bai-toan-cv-loi, C/16-thi-giac-3d và C/17-mo-hinh-sinh, và đặt sẵn câu hỏi mà C/18-pretraining-va-foundation trả lời: nếu giả định phải mua bằng dữ liệu, thì mua ở đâu cho rẻ nhất. Hai nhánh ngang nên đọc ngay sau chương này. Thứ nhất là B/11-gioi-han-tinh-toan, để biến các phép tính FLOPs ở đây thành dự đoán độ trễ có căn cứ. Thứ hai là E/24-thi-nghiem-va-ablation, để không bị chính các bảng so sánh trong chương này dẫn tới kết luận quá mạnh.

Tổng kết chương

Ta đã đi qua ba mạch và về cùng một chỗ. Mạch 1 cho thấy dòng CNN là một chuỗi gỡ nút thắt, mỗi bước đẩy độ khó sang một trục khác, và mất xích cuối — ConvNeXt — buộc ta xét lại toàn bộ các so sánh trước đó. Mạch 2 cho thấy attention mua được đường đi thông tin ngắn nhất có thể bằng chi phí bậc hai, và ba hướng chống chi phí đó nằm ở ba tầng trừu tượng khác nhau. Mạch 3 cho thấy khi vứt bỏ giả định thị giác, cái giá hiện ra rất sạch dưới dạng lượng dữ liệu cần có — và có ít nhất ba loại tiền để trả khoản đó.

Còn bỏ ngỏ ba chỗ. Thứ nhất, chương này chỉ nói về *trích đặc trưng*; việc biến đặc trưng thành đầu ra của một bài toán cụ thể — hộp, mặt nạ, khoảng cách — là nội dung của C/14-bai-toan-cv-loi. Thứ hai, cách rẻ nhất để trả khoản nợ giả định là dùng một mô hình đã pretrain, và toàn bộ chủ đề đó nằm ở C/18-pretraining-va-foundation. Thứ ba, mô hình không gian trạng thái là hướng đang chuyển động nhanh nhất trong ba mạch. Chương này đánh giá rằng nó chưa thay thế

attention trong thị giác. Đó là đánh giá tính tới tháng 9/2026, không phải một kết luận bền. Đọc tiếp C/14-bai-toan-cv-loi với câu hỏi: khi lắp một head cụ thể lên các backbone này, ràng buộc nào của backbone sẽ trở thành ràng buộc của cả hệ thống?

■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Trục duy nhất của chương là “cài sẵn nhiều giả định, cần ít dữ liệu” đối lại “cài ít giả định, cần nhiều dữ liệu”. Hãy đặt ResNet, Swin và ViT phẳng lên trục đó, rồi nói cái gì trong mô hình thực sự thay đổi khi dữ liệu tăng, khiến thứ hạng đảo chiều?
2. Kết nối tắt của ResNet, cổng của LSTM và chuẩn hoá theo tầng cùng phục vụ một mục tiêu đã đặt ra ở C/12-co-che-huan-luyen. Mục tiêu đó là gì, và vì sao attention giải quyết nó theo một cách khác hẳn ba thứ kia?
3. Nếu ConvNeXt đúng khi nói recipe quan trọng hơn kiến trúc, thì các bảng so sánh trong chính chương này còn đọc được ở mức nào, và bạn đòi thêm cột nào trước khi tin một dòng?
4. FLOPs xuất hiện ở cả ba mục nhưng không mục nào coi nó là độ trễ. MobileNet, FlashAttention và attention của sổ mỗi cái phá vỡ quan hệ FLOPs–độ trễ theo kiểu riêng nào, và đại lượng chung nào giải thích được cả ba?
5. Bạn cần một backbone cho ảnh rất lớn, vật thể chỉ vài chục pixel, và ngân sách bộ nhớ chặt. Ràng buộc nào loại bớt phương án trước, và vì sao câu trả lời đôi khi bạn có hay không có một mô hình pretrain?
6. Cả ba mục đều kể lịch sử như một chuỗi gỡ nút thắt. Nếu bạn tin nút thắt hiện tại nằm ở dữ liệu chứ không ở kiến trúc, thì quan sát nào trong chính chương này ủng hộ niềm tin đó, và quan sát nào phản bác?

Nguồn

- [1] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [3] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [4] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [5] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [6] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [7] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [8] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [9] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [10] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [11] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [12] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [13] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [14] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [15] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [16] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [17] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [18] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [19] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [20] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [21] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [22] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.

- [23] M. Oquab, T. Darcet, T. Moutakanni, et al., "DINOv2: Learning Robust Visual Features Without Supervision," *TMLR*, 2024.